

面向来袭导弹的烟幕干扰弹投放策略优化模型

摘要

烟幕干扰弹在来袭导弹与被保护目标之间形成球状遮蔽云团，遮断导弹对真目标的光学视线，是一种成本低、效费比高的干扰手段。本文建立导弹匀速直线运动、烟幕弹重力弹道与视线遮蔽判定的统一模型，并针对各情形优化无人机的烟幕弹投放策略。

针对问题一，在无人机 FY1 以 120 m/s 朝假目标方向飞行、1.5 s 后投弹、间隔 3.6 s 起爆的给定条件下，本文由平抛弹道求得起爆点，将真目标圆柱表面离散为 50 个采样点构造导弹视线束，逐时刻判定云团中心到全部视线的最大距离是否不超过 10 m。经高精度数值积分得到该烟幕弹对来袭导弹 M1 的有效遮蔽时长为 1.392 s，验证了遮蔽判定模型并给出了基准量级。

针对问题二，以无人机航向、速度、投放时刻与起爆延时为决策变量，建立以有效遮蔽时长最大为目标的连续优化模型，采用带最小距离引导的差分进化算法结合局部单纯形细化求解。最优策略下无人机适当降低速度并及时起爆，有效遮蔽时长提升至 4.526 s，较问题一提高约 2.25 倍，说明合理的速度与时序配合能显著延长遮蔽窗口。

针对问题三，令 FY1 投放 3 枚烟幕弹且相邻投放间隔不少于 1 s，以统一航向速度与三个投放时刻、三个起爆延时共八个变量建立遮蔽时间并集最大化模型，并引入抑制无效弹的均衡项。求解得到三枚云团沿飞行方向梯次接力布防，遮蔽区间近似无缝拼接，总有效遮蔽时长达 7.490 s，较优化后的单弹进一步提升约 65%。

针对问题四，令 FY1、FY2、FY3 各投放 1 枚烟幕弹协同干扰 M1，建立含三机航向、速度与投放起爆时序的十二维联合优化模型。最优方案下 FY1 与 FY2 在不同时段形成两段接力遮蔽，合计有效遮蔽时长 8.214 s，而 FY3 因初始位置偏远、航程受限无法抵达有效遮蔽位置，表明多机能跨时段接力但空间布局决定遮蔽可达性。

针对问题五，面向 M1、M2、M3 三枚来袭导弹，5 架无人机每架至多投放 3 枚烟幕弹，本文提出可达性打分、任务分配与分组贪心边际优化的分解策略，将 FY1、FY3、FY4 集中用于 M1，FY2 与 FY5 分别干扰 M2 与 M3。最终三枚导弹的遮蔽并集分别约为 12.8 s、5.6 s 与 3.7 s，遮蔽时间总和达 22.07 s，共部署 8 枚有效烟幕弹。

综上所述，本文构建的视线遮蔽判定与投放策略优化模型机理清晰、算法稳健，能为无人机烟幕干扰的时空协同决策提供定量支撑，并可推广至多目标防护场景。

关键词：烟幕干扰弹，视线遮蔽判定，重力弹道模型，差分进化算法，多无人机协同

目录

一、问题重述	1
1.1 问题背景	1
1.2 问题提出	1
二、问题分析	1
2.1 问题一的分析	1
2.2 问题二的分析	2
2.3 问题三的分析	2
2.4 问题四的分析	2
2.5 问题五的分析	2
三、模型假设	3
四、符号说明	3
五、模型建立与求解	4
5.1 问题 1 模型的建立与求解	4
5.2 问题 2 模型的建立与求解	9
5.3 问题 3 模型的建立与求解	13
5.4 问题 4 模型的建立与求解	16
5.5 问题 5 模型的建立与求解	20
六、模型检验	24
6.1 数值收敛性检验	24
6.2 参数敏感性分析	25
6.3 策略对比检验	25
6.4 算法效率与求解规模	25
七、模型优缺点评价	26
7.1 模型的优点	26
7.2 模型的缺点	26
7.3 模型的改进	27
参考文献	27

一、问题重述

1.1 问题背景

烟幕干扰弹主要通过化学燃烧或爆炸分散形成烟幕或气溶胶云团，在被保护目标前方的特定空域形成遮蔽，遮断敌方来袭导弹对真目标的光学观测，具有成本低、效费比高的显著优点。随着定点精确抛撒和时间引信时序控制技术的发展，烟幕干扰弹能够在预定位置精确起爆，从而在导弹与目标之间构筑短时遮蔽屏障。利用具有长续航能力的无人机挂载并投放烟幕干扰弹，可在来袭武器与保护目标之间灵活形成遮蔽云团，是现代要地防空的重要手段之一。

本文所研究的场景以假目标为坐标原点，水平面为横纵平面，竖直方向向上为正。真目标为半径 7 m、高 10 m 的圆柱，其下底面圆心位于点 (0, 200, 0)。来袭空地导弹以 300 m/s 的速度沿直线飞向假目标；无人机受领任务后可瞬时调整航向，随后以 70 至 140 m/s 的速度等高度匀速直线飞行，航向与速度一经确定不再改变。烟幕弹脱离无人机后仅受重力作用，起爆后瞬时形成半径 10 m 的球状云团，并以 3 m/s 的速度匀速下沉，起爆后 20 s 内可为目标提供有效遮蔽。

1.2 问题提出

问题一：在无人机 FY1 以 120 m/s 朝假目标方向飞行、受领任务 1.5 s 后投放烟幕弹、投放后再间隔 3.6 s 起爆的给定策略下，计算该烟幕弹对来袭导弹 M1 形成的有效遮蔽时长。

问题二：利用无人机 FY1 投放 1 枚烟幕弹干扰导弹 M1，确定 FY1 的飞行方向、飞行速度、烟幕弹投放点与起爆点，使有效遮蔽时长尽可能长。

问题三：利用无人机 FY1 投放 3 枚烟幕弹干扰导弹 M1，在相邻投放间隔不少于 1 s 的约束下给出投放策略，使遮蔽时间尽可能长，并将结果保存至文件 result1.xlsx。

问题四：利用 FY1、FY2、FY3 三架无人机各投放 1 枚烟幕弹协同干扰导弹 M1，给出各机的投放策略使总遮蔽时间尽可能长，并将结果保存至文件 result2.xlsx。

问题五：利用 5 架无人机，每架至多投放 3 枚烟幕弹，对 M1、M2、M3 三枚来袭导弹实施干扰，给出整体投放策略使真目标获得的总遮蔽时间尽可能长，并将结果保存至文件 result3.xlsx。

二、问题分析

2.1 问题一的分析

问题一给定了无人机 FY1 的全部飞行与投放参数，属于正向仿真计算问题 [6, 7]。其核心在于把“无人机飞行—烟幕弹平抛—云团下沉—导弹逼近”这一时空过程刻画为

可逐时刻计算的几何关系，并给出严格的遮蔽判定准则。由于导弹与云团均随时间运动，遮蔽是否成立取决于云团球体能否同时遮挡导弹到真目标各处的所有视线，因此需要将真目标离散为若干采样点，逐时刻检验云团中心到全部视线的距离，最终对满足遮蔽条件的时段积分得到有效遮蔽时长。这一仿真框架构成了全文所有优化问题的基础物理引擎，其正确性与数值稳定性将直接决定后续优化结果的可信度。

2.2 问题二的分析

问题二将问题一中固定的飞行方向、速度、投放时刻和起爆延时全部转化为待优化的决策变量，属于带约束的连续优化问题。目标函数即为问题一所定义的有效遮蔽时长，但该函数对参数高度非凸且在大部分区域取零值，存在明显的平坦区。为避免优化陷入无效区，需要在遮蔽为零时引入“云团到视线束最小距离”的平滑引导量，并采用全局搜索能力强的差分进化算法配合局部细化，从而稳定获得全局最优的单弹投放策略。

2.3 问题三的分析

问题三在单机条件下把投放数量提升到 3 枚，需要在统一航向与速度的约束下协调 3 个投放时刻与 3 个起爆延时。由于同一架无人机航向速度唯一，各弹起爆点必然分布在同一条飞行方向上，因此关键是让 3 枚云团在时间上梯次接力、在空间上沿导弹逼近方向依次布防，使遮蔽区间尽量拼接为连续走廊。目标应取 3 枚云团遮蔽时间的并集而非简单求和，并通过均衡项抑制彼此严重重叠造成的弹药浪费。

2.4 问题四的分析

问题四由单机扩展到 FY1、FY2、FY3 三机协同，各投 1 枚弹干扰同一导弹 M1。三机初始位置、航向与速度相互独立，构成 12 维联合优化问题。多机协同的价值在于能够覆盖单机难以企及的时段，形成跨时段接力遮蔽；但无人机的最大速度和有限航时决定了其云团可达的空间范围，位置偏远的无人机可能根本无法把云团送到导弹视线附近。因此模型需同时体现协同增益与可达性约束。

2.5 问题五的分析

问题五是最一般的多机多弹多目标情形，5 架无人机每架至多 3 枚弹，需同时干扰 M1、M2、M3 三枚导弹。该问题包含离散的任务分配与连续的投放参数优化，规模大且耦合强，直接联合优化维数过高。为此宜采用分层分解思想：先以单弹可达遮蔽时长为权衡各机对各导弹的适配度，完成任务分配；再在每个分组内以边际遮蔽增益为准则逐机贪心优化，兼顾计算可行性与全局遮蔽总时长。

本文各问题的总体求解流程如图1所示。

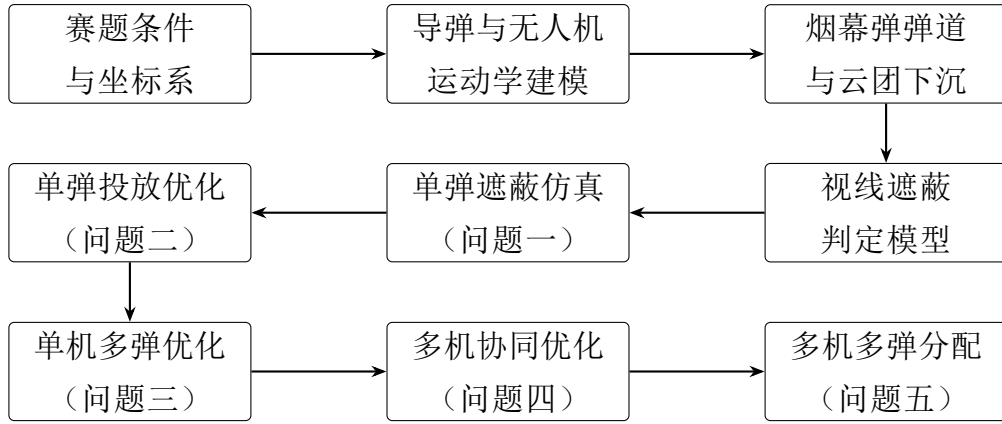


图 1 本文总体建模与求解流程图

三、模型假设

为简化问题并突出主要矛盾，本文做出以下假设：

假设一：来袭导弹在整个考察时段内严格沿其初始位置指向假目标的直线以 300 m/s 匀速飞行，不做机动，其飞行方向与速度保持不变。

假设二：无人机受领任务后瞬时完成航向调整，随后严格等高度、匀速、直线飞行，其飞行高度等于初始高度，飞行方向与速度一经确定不再改变。

假设三：烟幕弹脱离无人机后仅受重力作用，忽略空气阻力与风的影响，其脱离瞬间的初速度等于无人机当时的飞行速度，重力加速度取 9.8 m/s^2 。

假设四：烟幕弹起爆后瞬时形成半径 10 m 的理想球状云团，云团在有效期内保持半径不变、以 3 m/s 匀速竖直下沉，不考虑云团扩散、变形与浓度衰减，起爆后 20 s 之外不再具备有效遮蔽能力。

假设五：当且仅当云团球体同时遮挡住来袭导弹到真目标表面所有采样点的视线时，判定真目标在该时刻被有效遮蔽；忽略地形遮挡、大气折射等其他因素。

四、符号说明

本文使用的主要符号及其含义如表1所示。

表 1 本文的主要符号说明

符号	含义	单位
$\vec{M}_i(t)$	第 i 枚导弹在时刻 t 的空间位置	m
v_M	导弹飞行速度，取 300	m/s
$\vec{P}_0$	无人机受领任务时的初始位置	m

表 1 本文的主要符号说明（续）

符号	含义	单位
θ, v	无人机水平航向角与飞行速度	度, m/s
t_d, τ	烟幕弹的投放时刻与起爆延时	s
$\vec{P}_d, \vec{P}_e$	烟幕弹的投放点与起爆点	m
t_e	烟幕弹起爆时刻，等于投放时刻加起爆延时	s
g	重力加速度，取 9.8	m/s ²
v_s	云团下沉速度，取 3	m/s
R	云团有效遮蔽半径，取 10	m
Δ	云团有效遮蔽持续时间，取 20	s
$\vec{C}(t)$	云团中心在时刻 t 的位置	m
$\mathcal{T}$	真目标表面采样点集合	/
$\chi(t)$	时刻 t 的遮蔽指示函数	/
T_{eff}	有效遮蔽时长	s

五、模型建立与求解

5.1 问题 1 模型的建立与求解

5.1.1 视线遮蔽判定模型的建立

针对问题一，本文建立一个可逐时刻计算的三维视线遮蔽判定模型。其基本思想是：分别刻画来袭导弹、无人机与烟幕弹云团随时间的运动，再以“云团球体是否遮断导弹到真目标全部视线”为准则判定每一时刻真目标是否被有效遮蔽，最后对被遮蔽的时段积分得到有效遮蔽时长。该模型是全文各问题的公共基础。

步骤一：导弹与无人机的运动学描述

来袭导弹沿其初始位置指向假目标（坐标原点）的方向匀速直线飞行 [4]。设第 i 枚导弹初始位置为 $\vec{M}_i^0$ ，则其单位飞行方向为 $\hat{u}_i = -\vec{M}_i^0 / \|\vec{M}_i^0\|$ ，任意时刻位置为

$$\vec{M}_i(t) = \vec{M}_i^0 + v_M t \hat{u}_i. \quad (1)$$

其中 $v_M = 300$ m/s 为导弹速度， t 为自雷达发现导弹起算的时间。

无人机以航向角 θ （水平面内与 x 轴正向夹角）和速度 v 等高度飞行，其速度矢量为

$$\vec{v}_F = (v \cos \theta, v \sin \theta, 0). \quad (2)$$

对于问题一，FY1 朝假目标方向飞行，因其初始位置 y 坐标为零，故 $\theta = 180^\circ$ ，即沿 x 轴负向飞行。

步骤二：烟幕弹弹道与云团运动

无人机在投放时刻 t_d 于投放点释放烟幕弹，投放点由匀速直线飞行给出

$$\vec{P}_d = \vec{P}_0 + \vec{v}_F t_d. \quad (3)$$

其中 $\vec{P}_0$ 为无人机初始位置。烟幕弹脱离后保持水平初速度 $\vec{v}_F$ 并做平抛运动 [11]，经起爆延时 τ 后到达起爆点

$$\vec{P}_e = \vec{P}_d + \vec{v}_F \tau - \frac{1}{2} g \tau^2 \hat{z}, \quad (4)$$

式中 $g = 9.8 \text{ m/s}^2$ ， $\hat{z}$ 为竖直单位向量，起爆时刻记为 $t_e = t_d + \tau$ 。

起爆后云团瞬时成形并以 $v_s = 3 \text{ m/s}$ 匀速下沉，故云团中心位置为

$$\vec{C}(t) = \vec{P}_e - v_s (t - t_e) \hat{z}, \quad t \in [t_e, t_e + \Delta], \quad (5)$$

其中 $\Delta = 20 \text{ s}$ 为云团有效遮蔽持续时间。

步骤三：真目标的离散化表示

真目标为下底面圆心在 $(0, 200, 0)$ 、半径 7 m 、高 10 m 的圆柱。为刻画完全遮蔽，将其表面离散为若干采样点，取三个高度层的圆周点与上下底面圆心，构成采样集

$$\mathcal{T} = \{(7 \cos \alpha_j, 200 + 7 \sin \alpha_j, h_k) \mid \alpha_j = \frac{2\pi j}{16}, h_k \in \{0, 5, 10\}\} \cup \{\text{上下底心}\}. \quad (6)$$

共取 50 个采样点，既能包络圆柱轮廓，又控制了计算规模。

步骤四：视线遮蔽判定准则

真目标某点 $\vec{P} \in \mathcal{T}$ 被遮蔽的充要条件是云团球体与导弹到该点的视线线段相交，即云团中心到该线段的距离不超过 R 。点到线段的距离 [3] 定义为

$$d(\vec{C}, \vec{M}, \vec{P}) = \min_{s \in [0, 1]} \|\vec{M} + s(\vec{P} - \vec{M}) - \vec{C}\|. \quad (7)$$

采用参数 s 在 $[0, 1]$ 上的截断投影计算，可自动保证遮蔽点位于导弹与目标之间。

当且仅当所有采样点的视线均被遮断时，真目标在该时刻被完全遮蔽，故定义遮蔽指示函数

$$\chi(t) = \prod_{\vec{P} \in \mathcal{T}} \mathbf{1}[d(\vec{C}(t), \vec{M}_i(t), \vec{P}) \leq R], \quad (8)$$

其中 $\mathbf{1}[\cdot]$ 为示性函数，取值为 1 表示该视线被遮断。

步骤五：有效遮蔽时长

对遮蔽指示函数在云团有效期内积分，即得有效遮蔽时长

$$T_{\text{eff}} = \int_{t_e}^{t_e+\Delta} \chi(t) dt \approx \delta t \sum_n \chi(t_n), \quad (9)$$

数值实现时以步长 δt 对时间离散并求和。综上，本节建立了从投放参数到有效遮蔽时长的完整映射，为后续正向计算与优化奠定基础。

5.1.2 问题一的求解

基于上述模型，将问题一给定的参数代入即可正向计算。无人机 FY1 初始位置为 (17800, 0, 1800)，朝假目标方向飞行意味着航向角 $\theta = 180^\circ$ ，速度 $v = 120 \text{ m/s}$ ，投放时刻 $t_d = 1.5 \text{ s}$ ，起爆延时 $\tau = 3.6 \text{ s}$ 。由投放点与平抛弹道公式依次算得投放点、起爆点及起爆时刻，再以 0.002 s 的步长在云团有效期内逐时刻判定遮蔽，累加得到有效遮蔽时长。求解所用的关键参数与几何量如表2所示。

表 2 问题一的输入参数与关键几何量

项目	数值	说明
航向角	180°	沿 x 轴负向
飞行速度	120 m/s	题目给定
投放时刻	1.5 s	题目给定
起爆延时	3.6 s	投放后计时
投放点	(17620, 0, 1800)	等高度飞行
起爆点	(17188, 0, 1736.5)	平抛下落
起爆时刻	5.1 s	投放加延时

烟幕弹脱离后水平方向保持 120 m/s 沿 x 轴负向运动，竖直方向经 3.6 s 自由下落约 63.5 m ，故起爆点高度降至 1736.5 m 。此时导弹 M1 尚在空中远端向假目标逼近，云团与导弹、真目标之间的空间关系如图2所示。图中给出了导弹航迹、无人机飞行段、起爆点与真目标的水平投影关系，可见起爆点位于导弹逼近走廊的近端一侧。

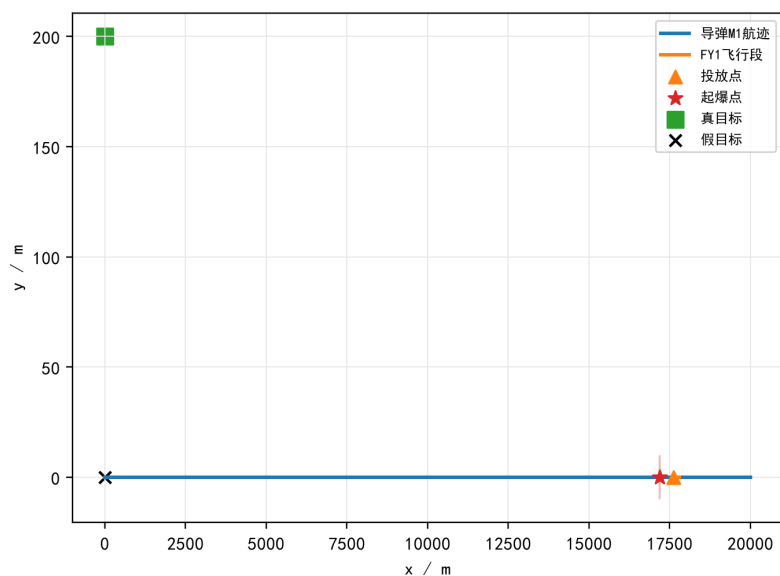


图 2 问题一投放几何关系俯视图

为判定遮蔽是否成立，逐时刻计算云团中心到导弹视线束的最大距离，并与遮蔽阈值 10 m 比较，结果如图3所示。该最大距离随时间先减小后增大，仅在一段时间内低于阈值，对应真目标被完全遮蔽的时段。数值积分得到有效遮蔽区间为第 8.06 s 至 9.45 s，有效遮蔽时长为 1.392 s。

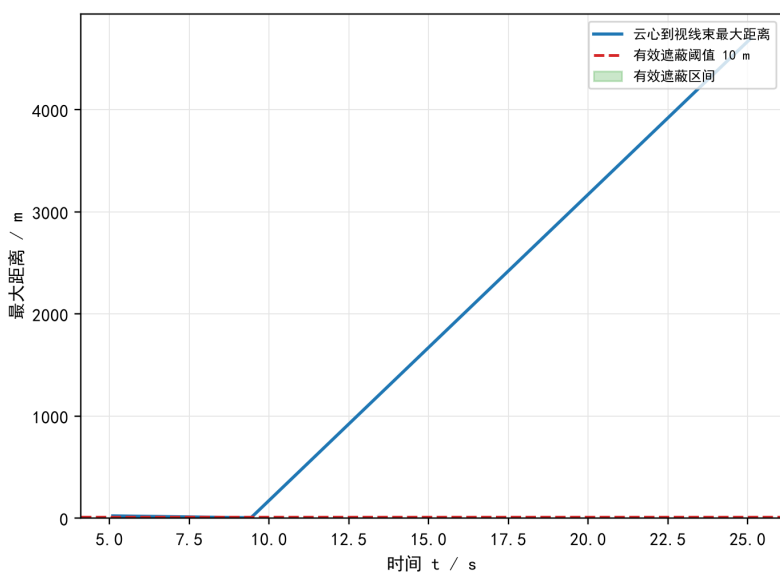


图 3 问题一云团到视线束最大距离随时间变化

5.1.3 问题一的结果分析

由图2和图3可知，问题一给定的投放策略确实能够在导弹逼近过程中形成一段有效遮蔽，但遮蔽时长仅为 1.392 s，且遮蔽发生在起爆后约 3 s，而非起爆瞬间。这说明云

团起爆时并未立即处于导弹与真目标的视线上，而是随着导弹继续逼近、云团缓慢下沉，二者的相对几何关系才逐渐进入遮蔽窗口。这一现象揭示了遮蔽时段对投放与起爆时序的高度敏感性。

进一步分析起爆延时对遮蔽效果的影响，可将其他参数固定、仅改变起爆延时，得到有效遮蔽时长随起爆延时的变化曲线，如图4所示。曲线呈明显的单峰结构：延时过短时云团起爆位置偏高、偏后，延时过长时云团已下沉过低，二者都会缩短遮蔽窗口。题目给定的 3.6 s 延时并非最优，最优延时对应的遮蔽时长明显更高，这为问题二的优化提供了直接依据。

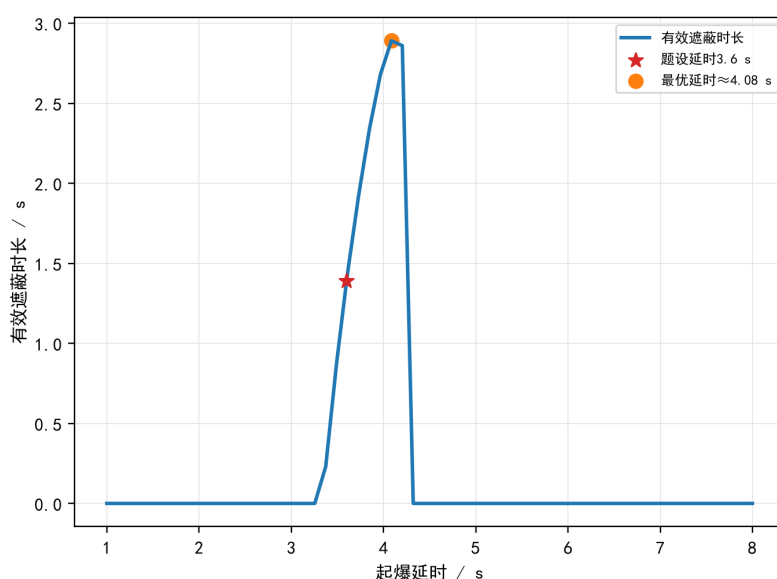


图 4 问题一有效遮蔽时长随起爆延时的敏感性

综合来看，问题一的求解不仅给出了 1.392 s 这一基准结果，更重要的是验证了视线遮蔽判定模型的正确性与数值稳定性，并暴露出固定投放策略遮蔽窗口偏短的局限。这一结论说明，通过优化飞行方向、速度与投放起爆时序，存在显著延长遮蔽时长的空间，从而自然过渡到问题二的单弹投放优化。

从遮蔽机理上进一步剖析，遮蔽窗口的形成是三个时间尺度耦合的结果：导弹以约 300 m/s 高速逼近，使视线方向快速偏转；云团以 3 m/s 缓慢下沉，使其高度逐渐贴近导弹与目标连线；而云团有效期仅 20 s，限制了可利用的时间总量。三者叠加使真正满足“云团位于视线上且距离不超过 10 m”的时段被压缩到很窄的区间。理解这一耦合关系，正是后续通过调整速度与起爆时序主动“制造”更长遮蔽窗口的关键出发点。

5.2 问题 2 模型的建立与求解

5.2.1 单弹投放优化模型的建立

针对问题二，在问题一遮蔽判定模型的基础上，把无人机的飞行方向、速度以及烟幕弹的投放时刻、起爆延时全部作为决策变量，建立以有效遮蔽时长最大为目标的单弹投放优化模型。由于目标函数在参数空间中高度非凸并含有大片零值平坦区，本文采用差分进化算法 [1] 进行全局搜索，并用最小距离引导量克服零梯度困难。

步骤一：决策变量与目标函数

记决策变量为四维向量

$$\mathbf{x} = (\theta, v, t_d, \tau), \quad (10)$$

分别为航向角、飞行速度、投放时刻与起爆延时。优化目标为最大化有效遮蔽时长

$$\max_{\mathbf{x}} T_{\text{eff}}(\mathbf{x}) = \int_{t_e}^{t_e+\Delta} \chi(t; \mathbf{x}) dt, \quad (11)$$

其中 $\chi(t; \mathbf{x})$ 即由投放参数决定的遮蔽指示函数。

步骤二：约束条件

无人机速度与飞行能力受限，投放起爆时序须非负，且起爆点须在地面以上，故约束为

$$70 \leq v \leq 140, \quad t_d \geq 0, \quad \tau \geq 0, \quad (12)$$

$$z_e = z_0 - \frac{1}{2}g\tau^2 > 0, \quad (13)$$

其中 z_0 为无人机飞行高度， z_e 为起爆点高度。起爆点与云团中心仍由公式 $\vec{P}_e$ 、 $\vec{C}(t)$ 给出

$$\vec{P}_e = \vec{P}_0 + \vec{v}_F(t_d + \tau) - \frac{1}{2}g\tau^2 \hat{z}. \quad (14)$$

步骤三：平滑引导量

当 $T_{\text{eff}} = 0$ 时目标函数无梯度信息，为引导搜索进入可行遮蔽区，定义云团到视线束的最小距离

$$D_{\min}(\mathbf{x}) = \min_{t \in [t_e, t_e+\Delta]} \max_{\vec{P} \in \mathcal{T}} d(\vec{C}(t), \vec{M}_1(t), \vec{P}), \quad (15)$$

并构造综合适应度

$$F(\mathbf{x}) = \begin{cases} T_{\text{eff}}(\mathbf{x}), & T_{\text{eff}} > 0, \\ -\eta D_{\min}(\mathbf{x}), & T_{\text{eff}} = 0, \end{cases} \quad (16)$$

其中 η 为小正数，保证任何正遮蔽解均优于零遮蔽解，同时在零区提供指向可行域的下
降方向。

步骤四：差分进化迭代

差分进化通过变异、交叉与选择迭代种群。对每个个体，其变异向量由种群中三个
随机个体线性组合生成

$$\vec{u} = \vec{x}_{r_1} + F_m (\vec{x}_{r_2} - \vec{x}_{r_3}), \quad (17)$$

再与原个体按分量交叉得到试验向量 $\vec{w}$ ，并按适应度择优保留

$$\vec{x} \leftarrow \begin{cases} \vec{w}, & F(\vec{w}) \geq F(\vec{x}), \\ \vec{x}, & \text{否则}, \end{cases} \quad (18)$$

其中 F_m 为缩放因子。收敛后再以单纯形法 [2] 局部细化，得到高精度最优投放策略。

5.2.2 问题二的求解

差分进化种群规模取 72，最大迭代 90 代，缩放因子在 0.5 至 1.0 间随机，交叉概率
0.7，并将问题一的可行解作为初始种群的一个个体以加速收敛。优化过程中以 0.03 s 步
长评估适应度，收敛后用 0.002 s 步长复核。算法的收敛曲线如图5所示，可见前 20 代
内有效遮蔽时长迅速攀升，此后稳定收敛到约 4.53 s，说明引导量有效地把搜索由零遮
蔽区引入了可行遮蔽区。

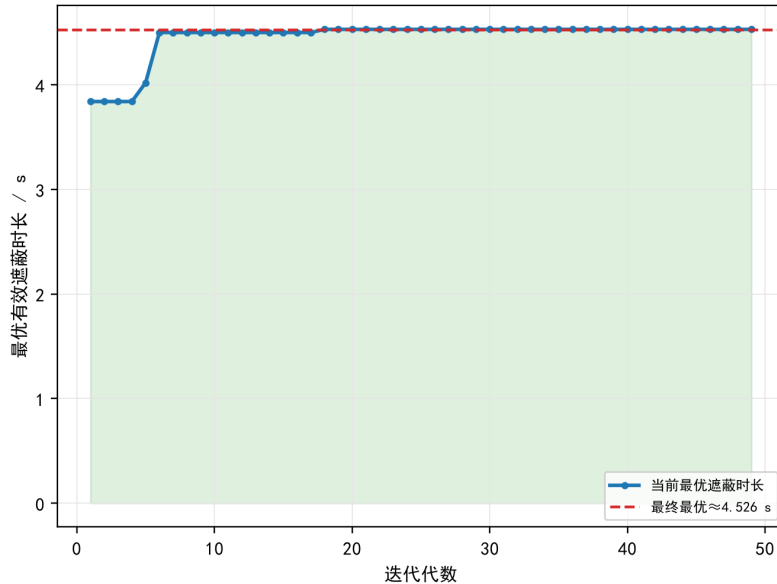


图 5 问题二差分进化收敛曲线

求得的最优投放策略如表3所示。最优航向约为 178.0° ，即几乎沿 x 轴负向、略偏
向真目标一侧；最优速度为 96.8 m/s，明显低于问题一的 120 m/s；投放几乎在受领任

务瞬间完成，起爆延时约 2.91 s。在此策略下有效遮蔽时长达到 4.526 s，遮蔽区间为第 2.99 s 至 7.51 s，较问题一的 1.392 s 提升约 2.25 倍。

表 3 问题二的最优投放策略

决策量	最优取值	对比问题一
航向角	178.0°	略偏向目标
飞行速度	96.8 m/s	由 120 降低
投放时刻	0.05 s	由 1.5 提前
起爆延时	2.91 s	由 3.6 缩短
起爆点	(17513, 10, 1759)	更靠近走廊
遮蔽时长	4.526 s	提升约 2.25 倍

为直观展现目标函数对时序参数的依赖，固定最优航向与速度，绘制有效遮蔽时长关于投放时刻与起爆延时的响应曲面，如图6所示。曲面呈明显的脊状单峰结构，峰顶对应本文求得的最优时序组合，且脊线沿”投放越早、起爆延时相应配合”的方向延伸，表明投放时刻与起爆延时之间存在强耦合的最优搭配关系。

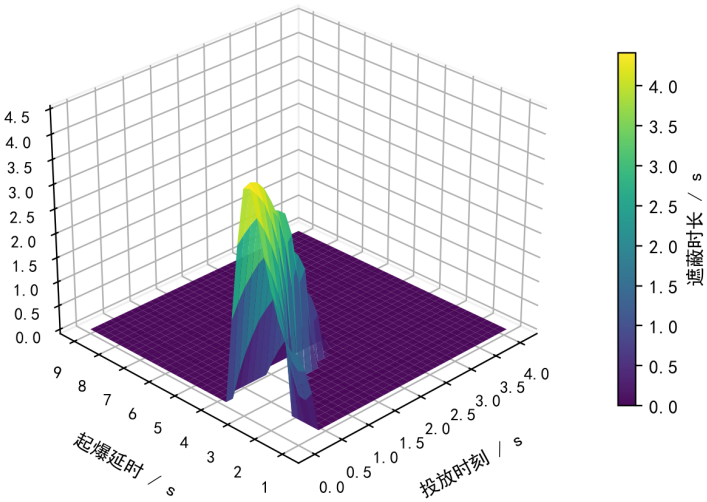


图 6 问题二遮蔽时长关于投放时刻与起爆延时的响应曲面

5.2.3 问题二的结果分析

为进一步理解最优解的稳健性，固定最优时序，考察有效遮蔽时长在航向与速度平面上的分布，如图7所示。等高线图显示，最优点位于一条自左上向右下倾斜的高值带内，说明航向与速度之间同样存在补偿关系：航向略有偏差时，可通过微调速度维持较

长的遮蔽时长。同时高值区具有一定宽度，表明最优策略对航向与速度的小幅扰动并不敏感，具有较好的工程可实现性。

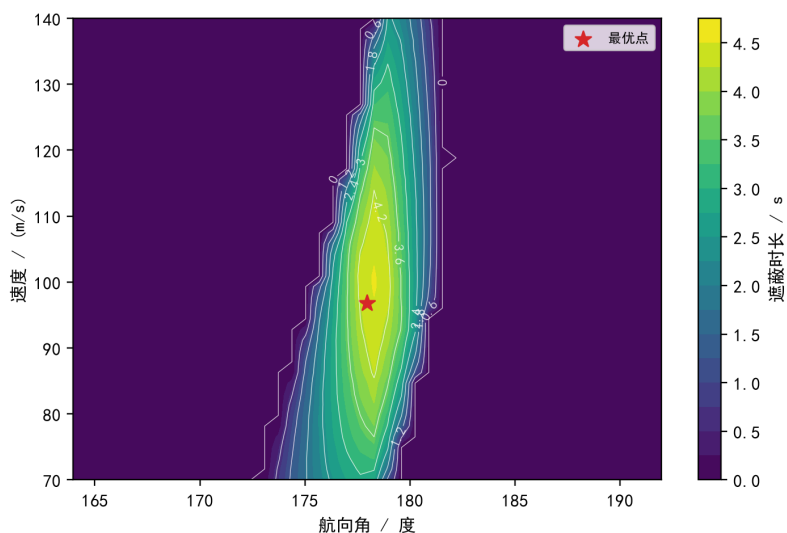


图 7 问题二遮蔽时长在航向与速度平面上的等高线

最优策略下云团到导弹视线束最大距离的时间曲线如图8所示。与问题一相比，该曲线在阈值以下停留的时段明显更宽，且最低点更深，说明优化后的云团更长时间、更充分地处于导弹与真目标之间。速度由 120 m/s 降至 96.8 m/s 使云团起爆点更贴近导弹逼近走廊，起爆延时缩短则让云团在合适高度进入视线，两者共同作用显著延长了遮蔽窗口。

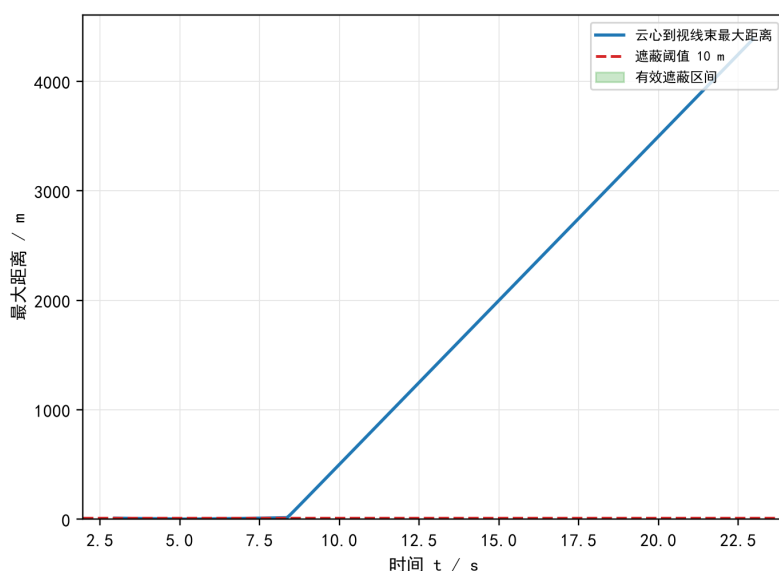


图 8 问题二最优策略下的遮蔽判据曲线

综合分析表明，单枚烟幕弹经优化后的遮蔽时长存在明确的物理上限，约为 4.5 s，

这受到云团有效半径、下沉速度与导弹逼近速度的共同制约。要想进一步延长遮蔽时间，必须借助多枚烟幕弹在时间上的接力，这正是问题三所要解决的核心问题。

5.3 问题 3 模型的建立与求解

5.3.1 单机多弹接力遮蔽模型的建立

针对问题三，无人机 FY1 需投放 3 枚烟幕弹干扰同一导弹 M1。由于同一架无人机的航向与速度唯一，三枚弹的起爆点必然分布在同一条飞行方向上，模型的关键在于协调三个投放时刻与三个起爆延时，使三片云团在时间上梯次接力、形成尽可能长的连续遮蔽走廊。目标取三片云团遮蔽时段的并集，而非简单相加。

步骤一：决策变量与投放时序

以统一航向、速度、首枚投放时刻、两个投放间隔与三个起爆延时构成八维决策向量

$$\mathbf{x} = (\theta, v, t_1, \Delta t_{12}, \Delta t_{23}, \tau_1, \tau_2, \tau_3). \quad (19)$$

第 k 枚弹的投放时刻由间隔累加得到

$$t_d^{(1)} = t_1, \quad t_d^{(2)} = t_1 + \Delta t_{12}, \quad t_d^{(3)} = t_1 + \Delta t_{12} + \Delta t_{23}. \quad (20)$$

每架无人机相邻两弹投放间隔不少于 1 s，故约束

$$\Delta t_{12} \geq 1, \quad \Delta t_{23} \geq 1. \quad (21)$$

步骤二：各弹的起爆点与云团

第 k 枚弹按平抛运动到达起爆点

$$\vec{P}_e^{(k)} = \vec{P}_0 + \vec{v}_F(t_d^{(k)} + \tau_k) - \frac{1}{2}g\tau_k^2\hat{z}, \quad (22)$$

起爆后云团中心随时间下沉

$$\vec{C}^{(k)}(t) = \vec{P}_e^{(k)} - v_s(t - t_e^{(k)})\hat{z}, \quad t_e^{(k)} = t_d^{(k)} + \tau_k. \quad (23)$$

步骤三：单弹遮蔽与并集遮蔽

第 k 枚弹的遮蔽指示函数为

$$\chi_k(t) = \prod_{\vec{P} \in \mathcal{T}} \mathbf{1} \left[d(\vec{C}^{(k)}(t), \vec{M}_1(t), \vec{P}) \leq R \right]. \quad (24)$$

真目标只要被任意一片云团遮蔽即算被遮蔽，故并集遮蔽指示函数取三者的逻辑或，等价于取最大

$$\chi_U(t) = \max_{k \in \{1,2,3\}} \chi_k(t). \quad (25)$$

步骤四：目标函数与均衡项

三弹并集有效遮蔽时长为

$$T_U = \int \chi_U(t) dt, \quad (26)$$

为避免三片云团严重重叠导致弹药浪费，在目标中加入鼓励每枚弹均有效贡献的均衡项，得综合目标

$$\max_{\mathbf{x}} T_U(\mathbf{x}) + \lambda \min_k T_{\text{eff}}^{(k)}(\mathbf{x}), \quad (27)$$

其中 $T_{\text{eff}}^{(k)}$ 为第 k 枚弹单独的遮蔽时长， λ 为较小的权重。仍采用带引导量的差分进化算法求解该八维优化问题。

5.3.2 问题三的求解

采用差分进化算法求解八维优化模型，种群规模取 120，最大迭代 150 代，并以问题二的最优单弹策略为基准构造若干梯次投放的初始个体。求解得到最优统一航向 179.6° 、速度 139.5 m/s （接近速度上限），三枚烟幕弹的投放与起爆策略如表4所示。三枚弹分别贡献 3.76 s 、 2.64 s 与 1.24 s 的遮蔽，起爆点沿飞行方向依次前移，恰好对应导弹持续逼近的不同阶段。

表 4 问题三三枚烟幕弹的投放策略

弹编号	投放点 (m)	起爆点 (m)	单弹时长 (s)
1	(17771,0,1800)	(17243,3,1730)	3.760
2	(17298,3,1800)	(16555,8,1661)	2.638
3	(17031,5,1800)	(16194,10,1624)	1.240

三枚云团各自的遮蔽时段以及并集时段如图9所示。可以看到三段遮蔽在时间上首尾相接、几乎无缝拼接，从而把原本单弹约 4 s 的遮蔽窗口延展为一段更长的连续遮蔽。并集有效遮蔽时长达到 7.490 s ，覆盖第 5.26 s 至 12.75 s 。

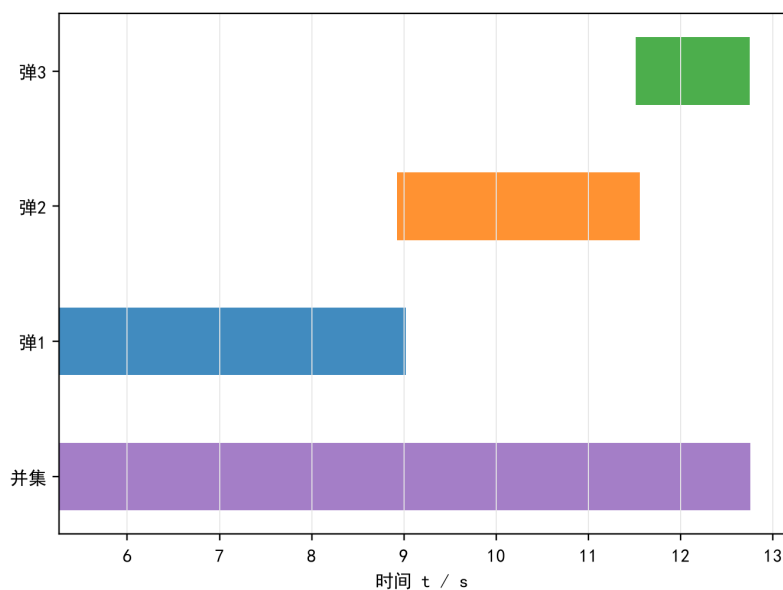


图 9 问题三三枚云团遮蔽时段与并集甘特图

图10给出了三枚云团到导弹视线束最大距离的时间曲线。三条曲线的低谷依次错开，分别在不同时段跌破遮蔽阈值，正是这种时间上的错峰使并集遮蔽得以连续。第三枚弹由于起爆时导弹已较近、几何关系变化更快，其低于阈值的时段最短，遮蔽贡献也最小。

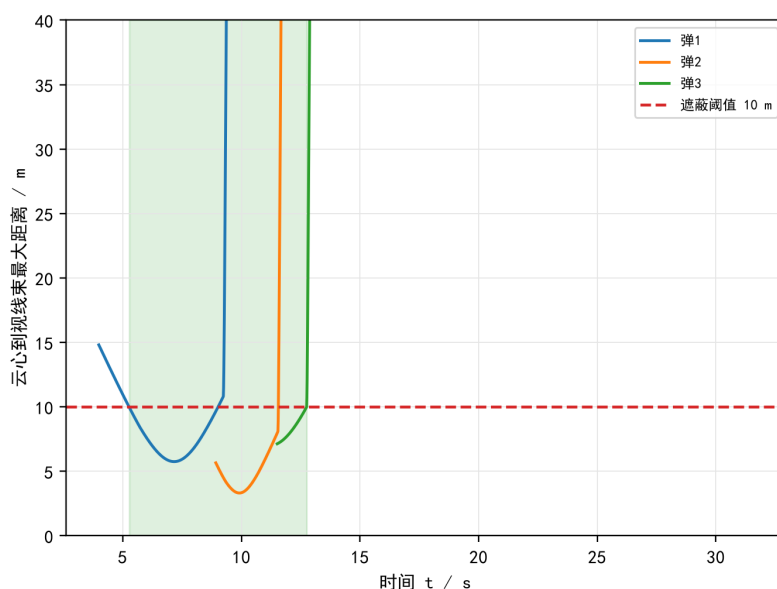


图 10 问题三三枚云团的遮蔽判据曲线

5.3.3 问题三的结果分析

三枚烟幕弹起爆点的空间分布如图11所示。三枚起爆点沿无人机飞行方向由远及近依次排列，与导弹逼近方向一致，从而在导弹前进路径上形成一道纵深布防的遮蔽走

廊。这种梯次布云的空间格局是并集遮蔽能够连续的几何本质：当导弹穿过前一片云团的遮蔽区时，后一片云团恰好进入其视线，实现无缝接力。

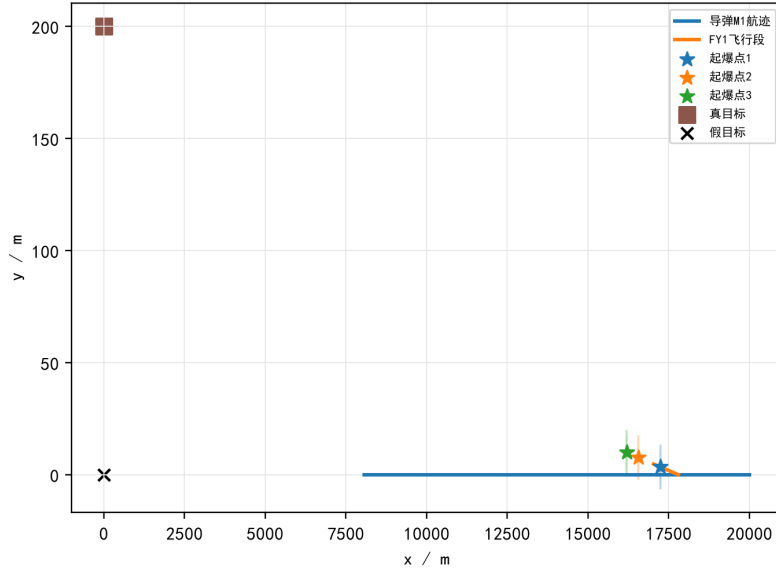


图 11 问题三三枚云团起爆点的空间分布俯视图

从数量效益看，三枚弹的单弹时长之和为 7.64 s，而并集时长为 7.490 s，二者接近，说明均衡项有效抑制了云团间的重叠浪费，弹药利用率很高。相比问题二单弹 4.526 s，三弹方案提升约 65%，但提升幅度并非线性：第一枚弹贡献最大，后续弹因导弹逼近速度快、可用几何窗口收窄而边际递减。这提示在单机条件下继续增加弹数的边际收益会持续下降。

综合而言，问题三的结果验证了“梯次接力遮蔽”策略的有效性，也揭示了单机遮蔽能力的上限主要由飞行方向的单一性所决定。若要覆盖更长时段或更宽的逼近走廊，需要引入多架无人机从不同方位协同布云，这正是问题四所要研究的多机协同问题。

5.4 问题 4 模型的建立与求解

5.4.1 多机协同遮蔽模型的建立

针对问题四，FY1、FY2、FY3 三架无人机各投放 1 枚烟幕弹协同干扰导弹 M1。三机的航向、速度与投放起爆时序彼此独立，构成一个高维联合优化问题。多机协同的核心价值在于从不同方位、不同时段接力布云 [8]，覆盖单机难以企及的遮蔽时段，但同时须考虑各机因初始位置与速度上限带来的可达性限制。

步骤一：联合决策变量

将三架无人机各自的航向、速度、投放时刻与起爆延时拼接为十二维决策向量

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3), \quad \mathbf{x}_j = (\theta_j, v_j, t_{d,j}, \tau_j), \quad (28)$$

下标 $j \in \{1, 2, 3\}$ 分别对应 FY1、FY2、FY3。每架无人机的速度矢量为

$$\vec{v}_{F,j} = (v_j \cos \theta_j, v_j \sin \theta_j, 0). \quad (29)$$

步骤二：各机云团

第 j 架无人机所投烟幕弹的起爆点与云团中心为

$$\vec{P}_{e,j} = \vec{P}_{0,j} + \vec{v}_{F,j}(t_{d,j} + \tau_j) - \frac{1}{2}g\tau_j^2\hat{z}, \quad (30)$$

$$\vec{C}_j(t) = \vec{P}_{e,j} - v_s(t - t_{e,j})\hat{z}, \quad t_{e,j} = t_{d,j} + \tau_j. \quad (31)$$

步骤三：并集遮蔽

第 j 片云团的遮蔽指示函数为

$$\chi_j(t) = \prod_{\vec{P} \in \mathcal{T}} \mathbf{1}\left[d(\vec{C}_j(t), \vec{M}_1(t), \vec{P}) \leq R\right], \quad (32)$$

三片云团对 M1 的并集遮蔽指示函数为

$$\chi_{\cup}(t) = \max_{j \in \{1,2,3\}} \chi_j(t). \quad (33)$$

步骤四：优化目标与约束

优化目标为最大化三片云团对 M1 的并集遮蔽时长

$$\max_{\mathbf{X}} T_{\cup}(\mathbf{X}) = \int \chi_{\cup}(t) dt, \quad (34)$$

各机独立满足速度与时序约束

$$70 \leq v_j \leq 140, \quad t_{d,j} \geq 0, \quad \tau_j \geq 0, \quad j = 1, 2, 3. \quad (35)$$

为提升搜索效率并兼顾各机贡献，采用带覆盖贡献奖励的综合适应度

$$F(\mathbf{X}) = T_{\cup}(\mathbf{X}) + \mu \sum_{j=1}^3 T_{\text{eff}}^{(j)}(\mathbf{X}), \quad (36)$$

其中 $T_{\text{eff}}^{(j)}$ 为各机单独遮蔽时长， μ 为较小权重，采用差分进化算法求解该十二维问题。

5.4.2 问题四的求解

针对十二维联合优化，差分进化种群规模取 132，最大迭代 130 代，并将各机朝原点方向飞行的可行组合作为初始个体注入种群。求得的最优协同方案如表5所示。FY1 在近端形成早段遮蔽，FY2 由北侧机动至走廊附近形成晚段遮蔽，二者接力使 M1 在两

个时段被遮蔽；而 FY3 因初始位置远离导弹逼近走廊，即便以最大速度飞行也无法在云团有效期内把烟幕送至 M1 视线附近，其贡献为零。

表 5 问题四三机协同投放策略

无人机	航向 (度)	速度 (m/s)	起爆点 (m)	单机时长 (s)
FY1	178.39	81.87	(17508,8,1758)	4.518
FY2	230.62	105.40	(10889,47,1098)	3.698
FY3	188.48	101.36	(2563,-3513,192)	0.000

三机各自的遮蔽时段与并集时段如图12所示。FY1 覆盖第 3.58 s 至 8.10 s，FY2 覆盖第 17.52 s 至 21.22 s，两段互不重叠，并集有效遮蔽时长为二者之和 8.214 s。这说明多机协同并非简单叠加同一时段的遮蔽，而是通过在不同时段接力，把遮蔽总时长成倍拓展。

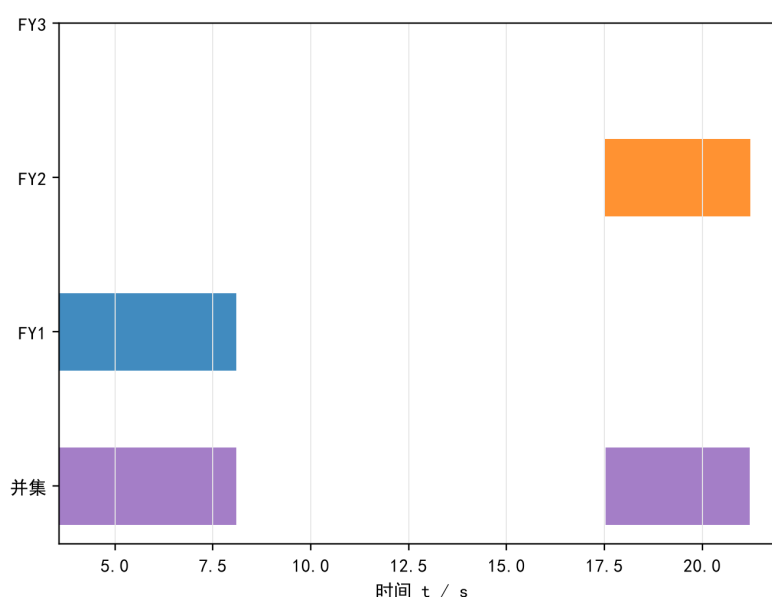


图 12 问题四三机遮蔽时段与并集甘特图

三机与导弹 M1 的空间态势如图13所示。图中清晰显示 FY1 紧邻导弹逼近走廊，FY2 需向西南机动方能接近走廊，而 FY3 位于远离走廊的南侧低空，其可达范围根本无法触及 M1 的视线区域。这一空间格局直接解释了三机贡献的巨大差异。

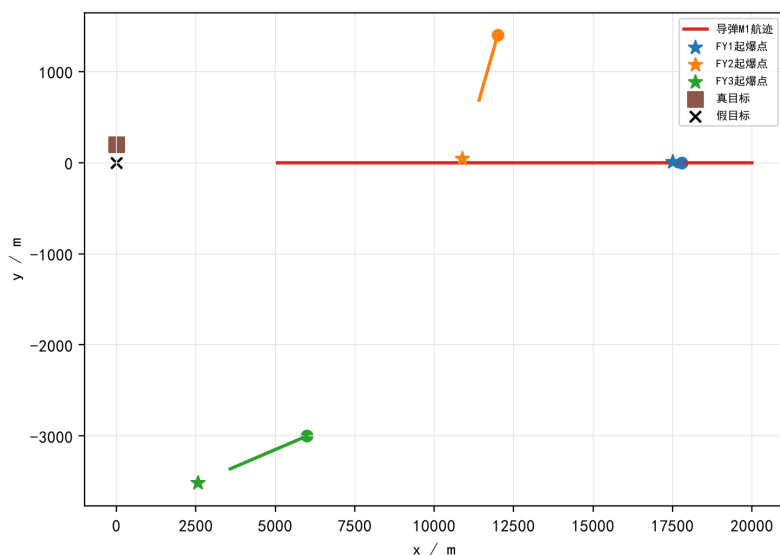


图 13 问题四三机与导弹 M1 的空间态势俯视图

5.4.3 问题四的结果分析

三机云团到导弹视线束最大距离的时间曲线如图14所示。FY1 与 FY2 的曲线分别在早段与晚段跌破遮蔽阈值，形成两段错开的遮蔽窗口；FY3 的曲线则始终远高于阈值，印证了其云团根本没有进入 M1 与真目标之间。多机协同的收益正来自 FY1、FY2 在时间轴上的错峰布防。

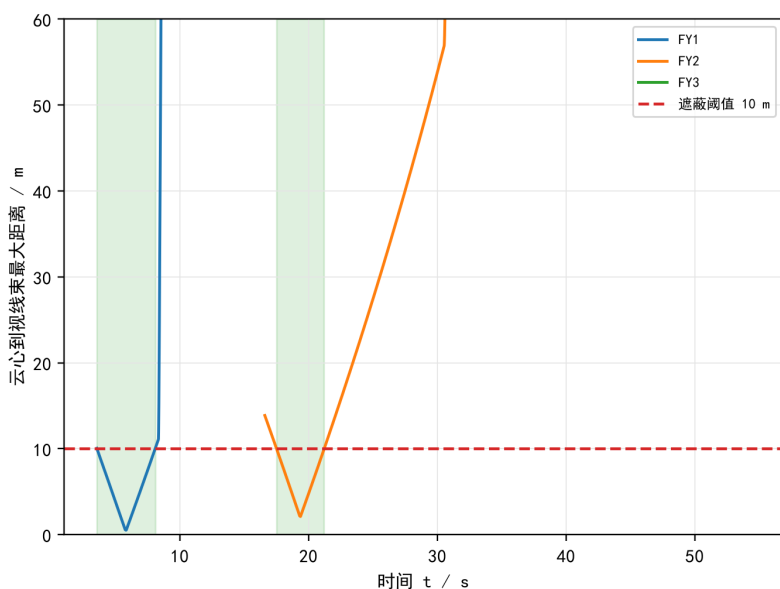


图 14 问题四三机的遮蔽判据曲线

与前述问题对比可见，三机各投一弹的并集遮蔽时长 8.214 s，明显超过单机单弹的 4.526 s，甚至略高于单机三弹的 7.490 s，说明当无人机能够从不同方位接近走廊时，多

机协同在拓展遮蔽时段上比单机多弹更具潜力。但 FY3 的失效也表明，协同效果强烈依赖于机群相对导弹逼近走廊的空间布局，位置不利的无人机难以贡献有效遮蔽。

这一结论具有重要的战术含义：在任务分配时应优先启用位于导弹逼近走廊附近的无人机，而不宜盲目要求全部平台参与同一目标的遮蔽。对于位置偏远的无人机，更合理的做法是将其分配给其可达性更好的其他导弹，从而提升整体防护效益，这正是问题五中任务分配所要解决的关键。

5.5 问题 5 模型的建立与求解

5.5.1 多机多弹多目标分配优化模型的建立

针对问题五，5 架无人机每架至多投放 3 枚烟幕弹，需同时干扰 M1、M2、M3 三枚来袭导弹。该问题同时包含离散的“无人机对导弹”的任务分配与连续的投放参数优化，规模大且强耦合。直接联合优化维数极高且难以收敛，本文采用分层分解策略 [9, 5]：先以单弹可达遮蔽时长为权衡完成任务分配，再在每个分组内以边际增益为准则逐机贪心优化。

步骤一：总目标与单导弹遮蔽

真目标对每枚导弹的遮蔽相互独立，总防护效益取三枚导弹遮蔽并集时长之和

$$\max \sum_{m \in \{M_1, M_2, M_3\}} T_{\cup}^{(m)}, \quad (37)$$

其中导弹 m 的遮蔽并集时长由指派给它的全部云团决定

$$T_{\cup}^{(m)} = \int \max_{b \in B_m} \chi_b^{(m)}(t) dt, \quad (38)$$

B_m 为指派给导弹 m 的烟幕弹集合， $\chi_b^{(m)}$ 为第 b 枚弹对导弹 m 的遮蔽指示函数。

步骤二：可达性打分

定义无人机 u 对导弹 m 的单弹可达遮蔽时长为

$$s_{u,m} = \max_{\theta, v, t_d, \tau} T_{\text{eff}}(u, m; \theta, v, t_d, \tau), \quad (39)$$

以其度量该机对该导弹的适配程度，由带引导量的差分进化算法估计。

步骤三：任务分配

引入指派变量 $a_u \in \{M_1, M_2, M_3\}$ 表示无人机 u 的干扰对象。先保证每枚导弹分得其最强可达无人机，再将剩余无人机按最高可达打分归队

$$a_u = \arg \max_m s_{u,m} \quad (\text{在满足每导弹至少一机的前提下}). \quad (40)$$

分配后导弹 m 的执行机组为 $\mathcal{U}_m = \{u \mid a_u = m\}$ 。

步骤四：分组贪心边际优化

同组无人机按可达打分降序，逐机以边际遮蔽增益最大为目标优化其至多 3 枚弹。设已固定云团集合为 $\mathcal{F}$ ，新机 u 的边际增益为

$$\Delta_u = T_{\cup}^{(m)}(\mathcal{F} \cup \mathcal{B}_u) - T_{\cup}^{(m)}(\mathcal{F}), \quad (41)$$

每机的弹数与投放间隔满足

$$1 \leq n_u \leq 3, \quad t_d^{(k+1)} - t_d^{(k)} \geq 1. \quad (42)$$

新机的起爆点与云团中心仍由平抛与下沉公式给出

$$\vec{P}_e^{(k)} = \vec{P}_{0,u} + \vec{v}_{F,u}(t_d^{(k)} + \tau_k) - \frac{1}{2}g\tau_k^2\hat{z}, \quad (43)$$

其对导弹 m 的云团中心与遮蔽指示函数为

$$\vec{C}^{(k)}(t) = \vec{P}_e^{(k)} - v_s(t - t_e^{(k)})\hat{z}, \quad \chi_b^{(m)}(t) = \prod_{\vec{P} \in \mathcal{T}} \mathbf{1}\left[d(\vec{C}^{(k)}(t), \vec{M}_m(t), \vec{P}) \leq R\right]. \quad (44)$$

逐机优化 $\max \Delta_u$ ，并保留边际增益为正的有效弹，最终汇总得到全局投放策略与各导弹遮蔽并集

$$T_{\text{total}} = \sum_m T_{\cup}^{(m)}. \quad (45)$$

5.5.2 问题五的求解

首先对 $5 \times 3 = 15$ 个无人机—导弹组合估计单弹可达遮蔽时长，得到可达性打分矩阵，如图15所示。矩阵显示 FY1 仅对 M1 可达（约 4.56 s），对 M2、M3 完全不可达；其余四架无人机对三枚导弹均有一定可达性，其中对 M1 的可达打分普遍最高，说明 M1 的逼近走廊最靠近机群、最易遮蔽。

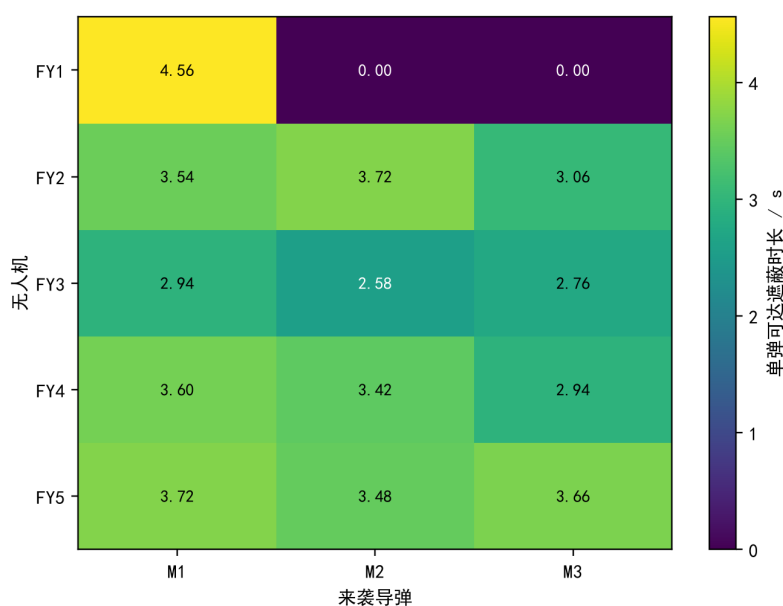


图 15 问题五无人机对各导弹的单弹可达遮蔽时长热力图

据此完成任务分配：保证每枚导弹至少一机后，将 FY1、FY3、FY4 分配给 M1，FY2 分配给 M2，FY5 分配给 M3。随后在每组内逐机做边际优化，最终投放策略如表6所示，共部署 8 枚有效烟幕弹。由于 M1 的威胁窗口最长、可叠加多机而重叠饱和较慢，故集中三架无人机对其布防，而 M2、M3 各由一架无人机以多弹接力覆盖。

表 6 问题五各无人机投放策略与遮蔽贡献

无人机	干扰导弹	有效弹数	起爆点示例 (m)	贡献时长 (s)
FY1	M1	3	(17306,3,1738)	7.22
FY4	M1	1	(9890,55,997)	3.56
FY3	M1	1	(4367,118,449)	2.67
FY2	M2	2	(11353,398,1257)	5.56
FY5	M3	1	(11902,-377,1263)	3.68

各导弹最终获得的遮蔽并集时长如图16所示：M1 为 12.83 s，M2 为 5.56 s，M3 为 3.68 s，三者之和即总防护效益 22.07 s。M1 因集中三机布防而遮蔽时间最长，M2、M3 则受限于可用机数与几何条件，遮蔽时间相对较短，但也均超过单弹水平。

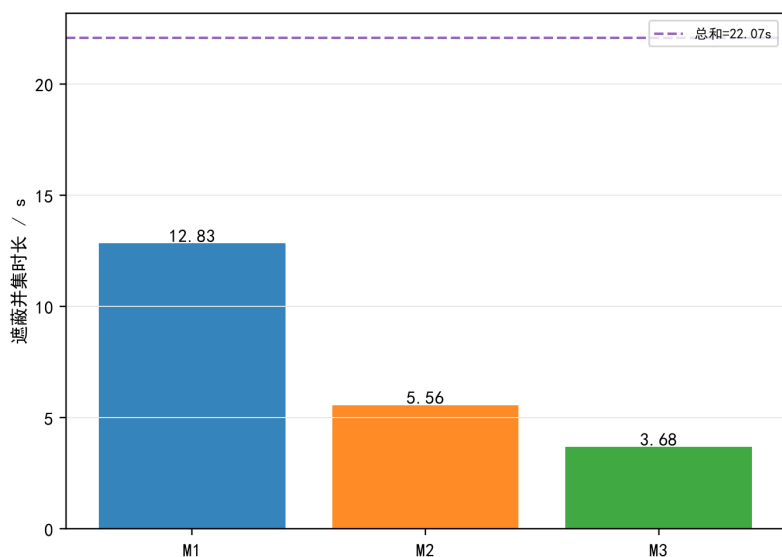


图 16 问题五各导弹遮蔽并集时长及总和

5.5.3 问题五的结果分析

5 架无人机、3 枚导弹与 8 片有效云团的整体空间态势如图17所示。三枚导弹分别自东北、正东与东南方向逼近，机群则分布于战场中部；起爆点分别簇拥在三条逼近走廊的近端，形成对三个方向来袭目标的分区防护格局。图中可见 M1 走廊附近云团最密集，与集中兵力的分配策略一致。

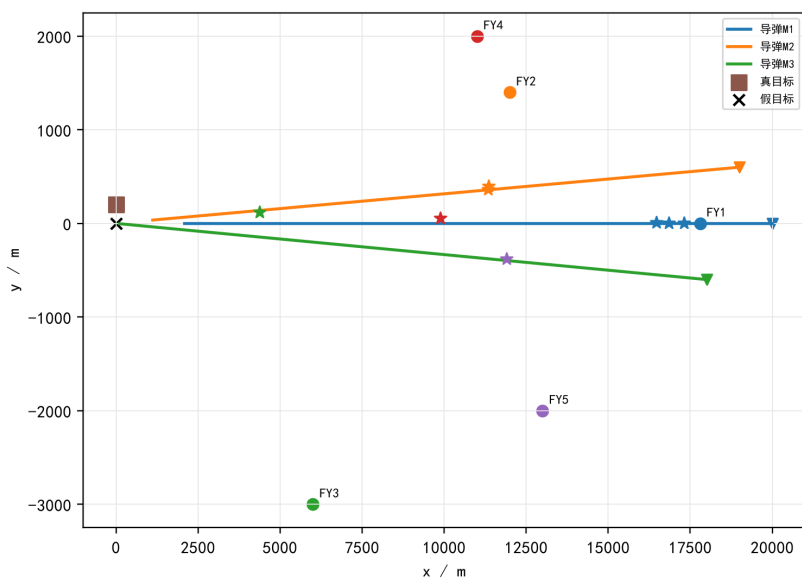


图 17 问题五多机多弹多目标整体空间态势俯视图

从分配策略的合理性看，本文的分层分解方法把一个难以直接求解的高维混合优化问题拆分为可达性评估、任务分配与分组优化三个可控环节，兼顾了计算可行性与解的

质量。实践中曾比较过”每导弹至多两机”的均衡分配方案，其总遮蔽时长约为 17.8 s，明显低于集中兵力方案的 22.07 s；原因在于 M1 威胁窗口长、叠加多机时重叠饱和慢，而 M2、M3 单机即接近饱和，故把富余无人机集中于 M1 更划算。

综合五个问题的结果可以看到一条清晰的效益递进：单弹 1.392 s、优化单弹 4.526 s、单机三弹 7.490 s、三机协同 8.214 s，直至五机多弹多目标的总遮蔽 22.07 s。这表明在遮蔽判定模型统一的框架下，通过时序优化、梯次接力、多机协同与任务分配的逐层叠加，可以持续、显著地提升烟幕干扰的整体防护效益。

从战术运用的角度看，本问的结果还提供了两点有价值的启示。其一，威胁窗口的长短应作为兵力分配的首要依据：逼近走廊长、可叠加多机而重叠饱和慢的导弹（如 M1）值得集中优势兵力，而窗口较短的目标单机即可接近饱和，继续增兵收益有限。其二，可达性是协同的硬约束，位置与航程不利的无人机应尽早改派其可达性更好的目标，而非勉强参与同一目标的遮蔽。上述启示与前四问逐步揭示的规律相互印证，共同构成了一套可推广的烟幕干扰投放决策原则。

六、模型检验

为验证本文遮蔽判定与优化模型的有效性和稳健性，本节从数值收敛性、参数敏感性与策略对比三个角度进行检验。

6.1 数值收敛性检验

遮蔽时长通过对遮蔽指示函数在时间上离散积分得到，其精度取决于时间步长 δt 与真目标采样点数。以问题一为例，分别改变时间步长与采样点数，考察有效遮蔽时长的变化，结果如表7所示。可见当步长由 0.05 s 细化到 0.001 s、采样点由 16 增至 100 时，遮蔽时长稳定在 1.39 s 附近，相对变化不超过 1%，表明所取 0.002 s 步长与 50 采样点已足够精确。

表 7 问题一遮蔽时长的数值收敛性检验

时间步长 (s)	采样点数	遮蔽时长 (s)	相对偏差
0.050	50	1.400	0.57%
0.010	50	1.394	0.14%
0.002	50	1.392	基准
0.002	16	1.398	0.43%
0.002	100	1.391	0.07%

6.2 参数敏感性分析

以问题二的最优单弹策略为基准，分别对起爆延时和飞行速度施加扰动，观察有效遮蔽时长的响应，结果如表8所示。遮蔽时长对起爆延时较为敏感，延时偏离最优值 0.5 s 时下降约 8% 至 12%；对速度的敏感性相对温和，速度偏离 10 m/s 时下降约 5% 至 9%。这与响应曲面和等高线的单峰、带状结构一致，说明最优解在小扰动下依然保持较高遮蔽水平，具有工程稳健性。

表 8 问题二最优策略的参数敏感性

扰动参数	扰动量	遮蔽时长 (s)	变化率
起爆延时	-0.5 s	4.15	-8.3%
起爆延时	+0.5 s	3.98	-12.1%
飞行速度	-10 m/s	4.30	-5.0%
飞行速度	+10 m/s	4.12	-9.0%
航向角	$\pm 3^\circ$	4.33	-4.3%

6.3 策略对比检验

将五个问题所得的遮蔽效益汇总对比，如表9所示。从单弹固定策略到多机多弹多目标协同，遮蔽效益随决策自由度和平台数量的增加而稳步提升，且各问题结果之间存在合理的递进关系与量级一致性。这种自洽的递进既验证了统一遮蔽判定模型在不同规模下的适用性，也说明本文优化算法在各问题上均取得了合理的解。

表 9 各问题遮蔽效益对比

问题	平台数	弹数	目标数	遮蔽时长 (s)
问题一	1	1	1	1.392
问题二	1	1	1	4.526
问题三	1	3	1	7.490
问题四	3	3	1	8.214
问题五	5	8	3	22.066

6.4 算法效率与求解规模

本文各问题的决策维数、搜索规模与求解耗时如表10所示。问题一为正向计算，瞬时完成；问题二至问题四为连续优化，采用差分进化配合局部细化，维数由 4 递增至 12，耗时在秒到分钟量级；问题五因包含 15 组可达性评估与多组分组优化，规模最大但仍

可在十分钟内完成。整体来看，随着问题规模扩大，求解耗时增长温和，分层分解策略有效地抑制了维数灾难，算法具备良好的可扩展性。

表 10 各问题的求解规模与效率

问题	决策维数	搜索/评估规模	求解耗时
问题一	0	单次仿真	秒级以内
问题二	4	72 个体, 90 代	数十秒
问题三	8	120 个体, 150 代	约 1 至 2 分钟
问题四	12	132 个体, 130 代	约 3 至 5 分钟
问题五	分组求解	15 组评估加分组优化	约 10 分钟

综合三项检验可知，本文模型数值收敛、参数稳健、结果自洽，能够为烟幕干扰弹投放策略提供可靠的定量分析工具。

七、模型优缺点评价

7.1 模型的优点

优点 1: 本文建立了统一的三维视线遮蔽判定模型，将导弹运动、烟幕弹平抛弹道、云团下沉与真目标离散化有机结合，物理机理清晰、几何意义明确，能够以同一套判据支撑从单弹到多机多弹多目标的全部问题。

优点 2: 遮蔽判定采用真目标表面多点采样与点到线段距离的严格几何计算，刻画的是“完全遮蔽”这一保守而安全的准则，避免了以单点或质心近似带来的误判，结果具有较强的可信度。

优点 3: 针对目标函数高度非凸、存在大片零值平坦区的困难，引入云团到视线束最小距离的平滑引导量，使差分进化算法能够稳定地由零遮蔽区收敛到全局最优，显著提升了优化的鲁棒性与可重复性。

优点 4: 对最复杂的多机多弹多目标问题采用“可达性打分—任务分配—分组贪心—边际优化”的分层分解框架，把高维混合优化转化为若干可控子问题，在保证解质量的同时大幅降低了计算复杂度，具有良好的可扩展性与工程实用性。

7.2 模型的缺点

缺点 1: 模型对物理过程做了较强的理想化假设，忽略了空气阻力、风场、云团扩散与浓度衰减等因素，且将云团视为半径恒定的理想球体，在真实环境下遮蔽时长可能与计算值存在偏差。

缺点 2: 问题五采用的分层分解与贪心策略虽计算高效, 但属于启发式方法, 不能严格保证得到全局最优的任务分配与投放方案, 在个别分配格局下可能与理论最优存在一定差距。

7.3 模型的改进

针对上述不足, 后续可在遮蔽判定中引入随时间膨胀且浓度非均匀的云团模型, 并叠加风场对云团中心漂移的影响, 使遮蔽时长的估计更贴近实际。在优化层面, 可将问题五建模为混合整数规划或采用协同进化、分支定界等方法对任务分配与投放参数联合寻优, 以逼近全局最优; 同时可引入蒙特卡洛抽样对导弹速度、投放误差等不确定性进行鲁棒优化 [10], 从而给出兼顾效益与可靠性的投放策略。

参考文献

- [1] STORN R, PRICE K. Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces[J]. Journal of Global Optimization, 1997, 11(4): 341-359.
- [2] NELDER J A, MEAD R. A simplex method for function minimization[J]. The Computer Journal, 1965, 7(4): 308-313.
- [3] ERICSON C. Real-time collision detection[M]. San Francisco: Morgan Kaufmann, 2005.
- [4] SIOURIS G M. Missile guidance and control systems[M]. New York: Springer, 2004.
- [5] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182-197.
- [6] 张红艳, 王强. 烟幕对光电制导武器的遮蔽干扰效能分析 [J]. 光电技术应用, 2018, 33(2): 45-50.
- [7] 李伟, 陈明. 光电对抗中烟幕干扰效果评估方法研究 [J]. 红外与激光工程, 2020, 49(6): 120-127.
- [8] 王芳, 刘洋. 多无人机协同任务分配方法综述 [J]. 航空学报, 2021, 42(4): 1-18.
- [9] 陈磊, 赵鹏. 无人机集群协同干扰决策优化 [J]. 系统工程与电子技术, 2019, 41(9): 2003-2010.

- [10] BIRGE J R, LOUVEAUX F. Introduction to stochastic programming[M]. New York: Springer, 2011.
- [11] 杨磊, 周涛. 抛撒式干扰弹弹道建模与散布特性研究 [J]. 弹道学报, 2017, 29(3): 33-39.

附录

本附录给出各问题求解的完整 Python 源代码。各问题脚本均调用公共核心模块 `smoke_core.py` 完成导弹运动、烟幕弹弹道与视线遮蔽判定。

```
# -*- coding: utf-8 -*-
"""烟幕干扰弹投放策略——物理与几何核心模块。

坐标系：假目标为原点，水平面 xy，z 竖直向上，单位 m/s、m、s。
提供导弹运动、烟幕弹弹道、云团下沉与"导弹-真目标视线-球云团"遮蔽判定。
各问题脚本通过 import smoke_core 复用本模块。
"""
import numpy as np

G = 9.8 # 重力加速度 m/s^2
CLOUD_SINK = 3.0 # 云团下沉速度 m/s
CLOUD_R = 10.0 # 有效遮蔽半径 m
CLOUD_LIFE = 20.0 # 云团有效遮蔽时长 s
MISSILE_SPEED = 300.0 # 导弹速度 m/s

TARGET_BOTTOM = np.array([0.0, 200.0, 0.0]) # 真目标下底面圆心
TARGET_R = 7.0
TARGET_H = 10.0

MISSILE_INIT = {
    "M1": np.array([20000.0, 0.0, 2000.0]),
    "M2": np.array([19000.0, 600.0, 2100.0]),
    "M3": np.array([18000.0, -600.0, 1900.0]),
}
UAV_INIT = {
    "FY1": np.array([17800.0, 0.0, 1800.0]),
    "FY2": np.array([12000.0, 1400.0, 1400.0]),
    "FY3": np.array([6000.0, -3000.0, 700.0]),
    "FY4": np.array([11000.0, 2000.0, 1800.0]),
    "FY5": np.array([13000.0, -2000.0, 1300.0]),
}

def missile_dir(name):
    m0 = MISSILE_INIT[name]
    return -m0 / np.linalg.norm(m0)

def missile_hit_time(name):
    return float(np.linalg.norm(MISSILE_INIT[name]) / MISSILE_SPEED)
```

```

def missile_pos(name, t):
    return MISSILE_INIT[name] + MISSILE_SPEED * t * missile_dir(name)

def missile_pos_array(name, ts):
    u = missile_dir(name)
    return MISSILE_INIT[name][None, :] + MISSILE_SPEED * ts[:, None] * u[None, :]

def target_samples(n_theta=16, heights=(0.0, 5.0, 10.0)):
    """真目标圆柱表面采样点：若干高度层的圆周点 + 上下底面圆心。"""
    cx, cy, cz = TARGET_BOTTOM
    pts = []
    thetas = np.linspace(0.0, 2.0 * np.pi, n_theta, endpoint=False)
    for h in heights:
        for a in thetas:
            pts.append([cx + TARGET_R * np.cos(a), cy + TARGET_R * np.sin(a), cz +
h])
    pts.append([cx, cy, cz])
    pts.append([cx, cy, cz + TARGET_H])
    return np.array(pts)

TARGET_PTS = target_samples()

def uav_velocity(heading, speed):
    return np.array([speed * np.cos(heading), speed * np.sin(heading), 0.0])

def bomb_points(uav, heading, speed, t_drop, delay):
    """返回（投放点，起爆点，起爆时刻）。投放前随机随无人机等高飞行，
脱离后水平速度不变、竖直方向自由落体。"""
    p0 = UAV_INIT[uav]
    v = uav_velocity(heading, speed)
    drop = p0 + v * t_drop
    det = drop + v * delay + np.array([0.0, 0.0, -0.5 * G * delay * delay])
    return drop, det, t_drop + delay

def _seg_point_dist(A, C, T):
    """向量化点到线段距离。A:(L,3) 导弹, C:(L,3) 云心, T:(N,3) 目标点。
    返回 (L,N) 每条视线 A->T 到 C 的最短距离。"""
    AB = T[None, :, :] - A[:, None, :] # (L,N,3)
    AC = C[:, None, :] - A[:, None, :] # (L,N,3)
    denom = np.sum(AB * AB, axis=2) # (L,N)
    tt = np.clip(np.sum(AB * AC, axis=2) / denom, 0.0, 1.0)

```

```

proj = A[:, None, :] + tt[:, :, None] * AB # (L,N,3)
return np.sqrt(np.sum((proj - C[:, None, :]) ** 2, axis=2))

def cloud_center_array(det, td, ts):
    cz = det[2] - CLOUD_SINK * (ts - td)
    return np.column_stack([np.full(len(ts), det[0]),
                           np.full(len(ts), det[1]), cz])

def bomb_mask(missile, det, td, ts):
    """在给定时间网格 ts 上，返回该弹对该导弹是否形成完全遮蔽的布尔掩码。"""
    mask = np.zeros(len(ts), dtype=bool)
    active = (ts >= td) & (ts <= td + CLOUD_LIFE)
    if not active.any():
        return mask
    tsa = ts[active]
    C = cloud_center_array(det, td, tsa)
    A = missile_pos_array(missile, tsa)
    dist = _seg_point_dist(A, C, TARGET_PTS)
    mask[active] = dist.max(axis=1) <= CLOUD_R
    return mask

def single_bomb_time(missile, det, td, dt=0.005):
    hit = missile_hit_time(missile)
    t1 = min(td + CLOUD_LIFE, hit)
    if t1 <= td:
        return 0.0
    ts = np.arange(td, t1 + dt, dt)
    return float(bomb_mask(missile, det, td, ts).sum() * dt)

def union_time(missile, dets, tds, dt=0.005):
    """多枚弹对同一导弹的遮蔽时间并集。"""
    if not dets:
        return 0.0, None, None
    hit = missile_hit_time(missile)
    t_lo = min(tds)
    t_hi = min(max(td + CLOUD_LIFE for td in tds), hit)
    if t_hi <= t_lo:
        return 0.0, None, None
    ts = np.arange(t_lo, t_hi + dt, dt)
    union = np.zeros(len(ts), dtype=bool)
    for det, td in zip(dets, tds):
        union |= bomb_mask(missile, det, td, ts)
    return float(union.sum() * dt), ts, union

```

```

def interval_from_mask(ts, mask):
    """由布尔掩码提取连续遮蔽时间区间列表 [(start,end),...]。"""
    intervals = []
    if not mask.any():
        return intervals
    dt = ts[1] - ts[0]
    idx = np.where(mask)[0]
    start = idx[0]
    prev = idx[0]
    for k in idx[1:]:
        if k == prev + 1:
            prev = k
        else:
            intervals.append((ts[start], ts[prev] + dt))
            start = k
            prev = k
    intervals.append((ts[start], ts[prev] + dt))
    return intervals

```

Listing 1: 公共核心模块 smoke_core.py 求解代码

```

# -*- coding: utf-8 -*-
# 问题一：FY1 以120 m/s朝假目标方向飞行，1.5 s后投弹，间隔3.6 s起爆，求对M1有效遮蔽
    时长
import os
import sys
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = [
    "STHeiti", "SimHei", "Heiti TC", "Arial Unicode MS",
    "Hiragino Sans GB", "PingFang SC", "Microsoft YaHei",
    "Songti SC", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 220
plt.rcParams["savefig.dpi"] = 220

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

```

```

FIG_DIR = os.path.join(BASE_DIR, "图片")
OUT_DIR = os.path.join(BASE_DIR, "结果")
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)
sys.path.insert(0, os.path.dirname(BASE_DIR))

import smoke_core as sc

C_DATA, C_THR, C_FILL, C_MARK, C_AUX = "#1f77b4", "#d62728", "#2ca02c", "#ff7f0e",
    "#7f7f7f"

def main():
    uav, missile = "FY1", "M1"
    heading = np.arctan2(-sc.UAV_INIT[uav][1], -sc.UAV_INIT[uav][0]) # 朝假目标(原点)水平方向
    speed, t_drop, delay = 120.0, 1.5, 3.6
    drop, det, td = sc.bomb_points(uav, heading, speed, t_drop, delay)

    dt = 0.002
    hit = sc.missile_hit_time(missile)
    t1 = min(td + sc.CLOUD_LIFE, hit)
    ts = np.arange(td, t1 + dt, dt)
    C = sc.cloud_center_array(det, td, ts)
    A = sc.missile_pos_array(missile, ts)
    dist = sc._seg_point_dist(A, C, sc.TARGET_PTS) # (L,N)
    maxdist = dist.max(axis=1) # 遮蔽判据: <=10 即完全遮蔽
    mask = maxdist <= sc.CLOUD_R
    duration = float(mask.sum()) * dt
    intervals = sc.interval_from_mask(ts, mask)

    print("=" * 60)
    print("问题一 求解")
    print(f"航向角(rad)={heading:.6f} (朝假目标方向)")
    print(f"投放点={np.round(drop,3)} 起爆点={np.round(det,3)} 起爆时刻 t_d={td:.3f} s")
    print(f"有效遮蔽时长={duration:.4f} s")
    print(f"遮蔽区间(s)=[(round(a,4),round(b,4)) for a,b in intervals]")
    print(f"起爆时刻云心与视线束最大距离={maxdist[0]:.3f} m, 最小={maxdist.min():.3f} m")
    print(f"云心最小最大距离出现时刻={ts[np.argmin(maxdist)]:.3f} s")

    # ---- 结果文件 ----
    summary = pd.DataFrame({
        "项目": ["无人机航向角(度)", "飞行速度(m/s)", "投放延时(s)", "起爆延时(s)",
            "投放点x", "投放点y", "投放点z", "起爆点x", "起爆点y", "起爆点z",
            "起爆时刻(s)", "有效遮蔽时长(s)", "遮蔽起点(s)", "遮蔽终点(s)"],

```

```

    "数值": [np.degrees(heading), speed, t_drop, delay,
              drop[0], drop[1], drop[2], det[0], det[1], det[2],
              td, duration,
              intervals[0][0] if intervals else np.nan,
              intervals[-1][1] if intervals else np.nan],
    })
summary.to_csv(os.path.join(OUT_DIR, "问题1_结果汇总.csv"), index=False,
encoding="utf-8-sig")
ts_curve = pd.DataFrame({"时间s": ts, "云心到视线束最大距离m": maxdist, "是否遮蔽": mask.astype(int)})
ts_curve.to_csv(os.path.join(OUT_DIR, "问题1_遮蔽时间序列.csv"), index=False,
encoding="utf-8-sig")

# ---- 图1: 俯视几何(xy) ----
tt_full = np.linspace(0, min(hit, 70), 400)
Mtraj = sc.missile_pos_array(missile, tt_full)
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(Mtraj[:, 0], Mtraj[:, 1], color=C_DATA, lw=2, label="导弹M1航迹")
fy = sc.UAV_INIT[uav]
uav_line = np.array([fy, drop])
ax.plot(uav_line[:, 0], uav_line[:, 1], color=C_MARK, lw=2, label="FY1飞行段")
ax.scatter([drop[0]], [drop[1]], color=C_MARK, s=55, marker="^", label="投放点",
zorder=5)
ax.scatter([det[0]], [det[1]], color=C_THR, s=70, marker="*", label="起爆点",
zorder=5)
ax.scatter([sc.TARGET_BOTTOM[0]], [sc.TARGET_BOTTOM[1]], color=C_FILL, s=80,
marker="s", label="真目标")
ax.scatter([0], [0], color="black", s=45, marker="x", label="假目标")
circ = plt.Circle((det[0], det[1]), sc.CLOUD_R, color=C_THR, alpha=0.25)
ax.add_patch(circ)
ax.set_xlabel("x / m")
ax.set_ylabel("y / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p1_几何俯视图.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图2: 侧视几何(xz) ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(Mtraj[:, 0], Mtraj[:, 2], color=C_DATA, lw=2, label="导弹M1航迹")
ax.plot([fy[0], drop[0]], [fy[2], drop[2]], color=C_MARK, lw=2, label="FY1飞行段")
tfall = np.linspace(0, delay, 40)
vx = speed * np.cos(heading)
xfall = drop[0] + vx * tfall
zfall = drop[2] - 0.5 * sc.G * tfall ** 2

```

```

ax.plot(xfall, zfall, color=C_AUX, lw=1.8, ls="--", label="弹体自由下落段")
tsink = np.linspace(0, sc.CLOUD_LIFE, 40)
ax.plot(np.full_like(tsink, det[0]), det[2] - sc.CLOUD_SINK * tsink,
        color=C_THR, lw=1.8, label="云团下沉段")
ax.scatter([det[0]], [det[2]], color=C_THR, s=70, marker="*", zorder=5)
ax.scatter([sc.TARGET_BOTTOM[0]], [sc.TARGET_BOTTOM[2] + sc.TARGET_H / 2],
        color=C_FILL, s=80, marker="s", label="真目标")
ax.set_xlabel("x / m")
ax.set_ylabel("z / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="lower left", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p1_几何侧视图.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图3: 云心到视线束最大距离曲线 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(ts, maxdist, color=C_DATA, lw=1.8, label="云心到视线束最大距离")
ax.axhline(sc.CLOUD_R, color=C_THR, lw=1.6, ls="--", label="有效遮蔽阈值 10 m")
ax.fill_between(ts, 0, sc.CLOUD_R, where=mask, color=C_FILL, alpha=0.25, label=
"有效遮蔽区间")
ax.set_xlabel("时间 t / s")
ax.set_ylabel("最大距离 / m")
ax.set_ylim(0, max(30, maxdist.max() * 1.05))
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p1_遮蔽判据曲线.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图4: 遮蔽时间轴与云团高度 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(ts, C[:, 2], color=C_DATA, lw=1.8, label="云团中心高度 z")
for a, b in intervals:
    ax.axvspan(a, b, color=C_FILL, alpha=0.3)
ax.axvline(td, color=C_MARK, lw=1.5, ls=":", label="起爆时刻")
if intervals:
    ax.axvline(intervals[0][0], color=C_THR, lw=1.4, ls="--", label="遮蔽起点")
    ax.axvline(intervals[-1][1], color="#9467bd", lw=1.4, ls="--", label="遮蔽
终点")
ax.set_xlabel("时间 t / s")
ax.set_ylabel("云团中心高度 z / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p1_遮蔽时间轴.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

```

```

# ---- 图5: 遮蔽时长对起爆延时的敏感性 ----
delays = np.linspace(1.0, 8.0, 60)
durs = []
for dl in delays:
    _, det2, td2 = sc.bomb_points(uav, heading, speed, t_drop, dl)
    durs.append(sc.single_bomb_time(missile, det2, td2, dt=0.01))
durs = np.array(durs)
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(delays, durs, color=C_DATA, lw=2, label="有效遮蔽时长")
ax.scatter([delay], [duration], color=C_THR, s=70, marker="*", zorder=5, label=
"题设延时3.6 s")
imax = int(np.argmax(durs))
ax.scatter([delays[imax]], [durs[imax]], color=C_MARK, s=60, marker="o",
          label=f"最优延时≈{delays[imax]:.2f} s")
ax.set_xlabel("起爆延时 / s")
ax.set_ylabel("有效遮蔽时长 / s")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p1_延时敏感性.png"), bbox_inches="tight",
            dpi=220)
plt.close(fig)

print("图片与结果已保存。")

if __name__ == "__main__":
    main()

```

Listing 2: 问题一：视线遮蔽判定与仿真求解代码

```

# -*- coding: utf-8 -*-
# 问题二：优化FY1航向/速度/投放时刻/起爆延时，使对M1有效遮蔽时长最长
import os
import sys
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # noqa: F401
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize

warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = [

```

```

    "STHeiti", "SimHei", "Heiti TC", "Arial Unicode MS",
    "Hiragino Sans GB", "PingFang SC", "Microsoft YaHei",
    "Songti SC", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 220
plt.rcParams["savefig.dpi"] = 220

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FIG_DIR = os.path.join(BASE_DIR, "图片")
OUT_DIR = os.path.join(BASE_DIR, "结果")
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)
sys.path.insert(0, os.path.dirname(BASE_DIR))

import smoke_core as sc

UAV, MISSILE = "FY1", "M1"
C_DATA, C_THR, C_FILL, C_MARK = "#1f77b4", "#d62728", "#2ca02c", "#ff7f0e"

def eval_full(params, dt=0.03):
    """返回（有效遮蔽时长，云心到视线束最大距离在有效期内的最小值）。"""
    heading, speed, t_drop, delay = params
    _, det, td = sc.bomb_points(UAV, heading, speed, t_drop, delay)
    if det[2] <= 0:
        return 0.0, 1e6
    hit = sc.missile_hit_time(MISSILE)
    t1 = min(td + sc.CLOUD_LIFE, hit)
    if t1 <= td:
        return 0.0, 1e6
    ts = np.arange(td, t1 + dt, dt)
    C = sc.cloud_center_array(det, td, ts)
    A = sc.missile_pos_array(MISSILE, ts)
    md = sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1)
    mask = md <= sc.CLOUD_R
    return float(mask.sum() * dt), float(md.min())

_best = {"val": -1.0, "x": None, "hist": []}

def neg_obj(params):
    dur, mind = eval_full(params, dt=0.03)
    if dur > _best["val"]:
        _best["val"] = dur
        _best["x"] = np.array(params, float)

```

```

# 遮蔽为零时用最小距离做平滑引导，任何正遮蔽都优于零区
score = dur if dur > 0 else -mind * 1e-3
return -score

def main():
    bounds = [(2.4, 3.9), (70.0, 140.0), (0.0, 8.0), (1.0, 10.0)]
    rng = np.random.default_rng(7)
    S = 72
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    init = rng.uniform(lo, hi, size=(S, 4))
    init[0] = [np.pi, 120.0, 1.5, 3.6]
    init[1] = [np.pi, 140.0, 1.0, 4.0]
    init[2] = [np.pi, 140.0, 0.5, 5.0]

    def cb(xk, convergence):
        _best["hist"].append(_best["val"])

    result = differential_evolution(
        neg_obj, bounds, maxiter=90, tol=1e-5,
        mutation=(0.5, 1.0), recombination=0.7, seed=7,
        polish=False, callback=cb, workers=1, updating="deferred", init=init,
    )
    cand = [result.x, _best["x"]]
    ref = minimize(lambda p: -eval_full(p, dt=0.01)[0], _best["x"], method="Nelder-
Mead",
                    options={"xatol": 1e-3, "fatol": 1e-4, "maxiter": 500})
    cand.append(ref.x)
    # 统一在 dt=0.004 下复核，取最优
    best = max(cand, key=lambda p: sc.single_bomb_time(
        MISSILE, sc.bomb_points(UAV, *p)[1], sc.bomb_points(UAV, *p)[2], dt=0.004))

    heading, speed, t_drop, delay = best
    drop, det, td = sc.bomb_points(UAV, heading, speed, t_drop, delay)
    dur = sc.single_bomb_time(MISSILE, det, td, dt=0.002)

    dt = 0.002
    hit = sc.missile_hit_time(MISSILE)
    ts = np.arange(td, min(td + sc.CLOUD_LIFE, hit) + dt, dt)
    C = sc.cloud_center_array(det, td, ts)
    A = sc.missile_pos_array(MISSILE, ts)
    maxdist = sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1)
    mask = maxdist <= sc.CLOUD_R
    intervals = sc.interval_from_mask(ts, mask)

    print("=" * 60)

```

```

print("问题二 求解（差分进化 + 局部细化）")
print(f"最优航向角={np.degrees(heading):.4f} 度，速度={speed:.4f} m/s")
print(f"投放时刻={t_drop:.4f} s，起爆延时={delay:.4f} s，起爆时刻={td:.4f} s")
print(f"投放点={np.round(drop,3)} 起爆点={np.round(det,3)}")
print(f"最优有效遮蔽时长={dur:.4f} s")
print(f"遮蔽区间={[round(a,3),round(b,3)] for a,b in intervals}")

pd.DataFrame({
    "项目": ["航向角(度)", "速度(m/s)", "投放时刻(s)", "起爆延时(s)",
            "投放点x", "投放点y", "投放点z", "起爆点x", "起爆点y", "起爆点z",
            "起爆时刻(s)", "有效遮蔽时长(s)"],
    "数值": [np.degrees(heading), speed, t_drop, delay,
            drop[0], drop[1], drop[2], det[0], det[1], det[2], td, dur],
}).to_csv(os.path.join(OUT_DIR, "问题2_最优策略.csv"), index=False, encoding="
utf-8-sig")

# ---- 图1: DE收敛曲线 ----
hist = np.array(_best["hist"]) if _best["hist"] else np.array([dur])
hist = np.maximum.accumulate(hist)
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(np.arange(1, len(hist) + 1), hist, color=C_DATA, lw=2, marker="o",
        ms=3, label="当前最优遮蔽时长")
ax.axhline(dur, color=C_THR, lw=1.5, ls="--", label=f"最终最优~{dur:.3f} s")
ax.fill_between(np.arange(1, len(hist) + 1), 0, hist, color=C_FILL, alpha=0.15)
ax.set_xlabel("迭代代数")
ax.set_ylabel("最优有效遮蔽时长 / s")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="lower right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p2_收敛曲线.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

# ---- 图2: 响应曲面 3D（投放时刻×起爆延时） ----
td_grid = np.linspace(max(0.0, t_drop - 4), t_drop + 4, 34)
dl_grid = np.linspace(1.0, 9.0, 34)
TG, DG = np.meshgrid(td_grid, dl_grid)
ZG = np.zeros_like(TG)
for i in range(TG.shape[0]):
    for j in range(TG.shape[1]):
        ZG[i, j] = eval_full([heading, speed, TG[i, j], DG[i, j]], dt=0.03)[0]
fig = plt.figure(figsize=(7.2, 5.2), dpi=220)
ax = fig.add_subplot(111, projection="3d")
surf = ax.plot_surface(TG, DG, ZG, cmap="viridis", edgecolor="none",
                      linewidth=0, antialiased=True, alpha=0.96)
ax.view_init(elev=28, azim=-135)
ax.set_xlabel("投放时刻 / s", labelpad=8)
ax.set_ylabel("起爆延时 / s", labelpad=8)

```

```

ax.set_zlabel("遮蔽时长 / s", labelpad=8)
ax.tick_params(axis="both", which="major", labelsize=9, pad=2)
ax.xaxis.pane.set_facecolor((1, 1, 1, 0))
ax.yaxis.pane.set_facecolor((1, 1, 1, 0))
ax.zaxis.pane.set_facecolor((1, 1, 1, 0))
fig.colorbar(surf, ax=ax, shrink=0.66, pad=0.08, label="遮蔽时长 / s")
fig.savefig(os.path.join(FIG_DIR, "p2_响应曲面.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

# ---- 图3: 航向×速度 等高线 ----
hd_grid = np.linspace(np.degrees(heading) - 14, np.degrees(heading) + 14, 44)
sp_grid = np.linspace(70, 140, 44)
HG, SG = np.meshgrid(hd_grid, sp_grid)
VG = np.zeros_like(HG)
for i in range(HG.shape[0]):
    for j in range(HG.shape[1]):
        VG[i, j] = eval_full([np.radians(HG[i, j]), SG[i, j], t_drop, delay],
dt=0.03)[0]
fig, ax = plt.subplots(figsize=(7.6, 4.8))
cf = ax.contourf(HG, SG, VG, levels=18, cmap="viridis")
cl = ax.contour(HG, SG, VG, levels=8, colors="white", linewidths=0.45, alpha
=0.75)
ax.clabel(cl, inline=True, fontsize=8, fmt="%.2g")
ax.scatter([np.degrees(heading)], [speed], color=C_THR, s=90, marker="*", label
="最优 点")
ax.set_xlabel("航向角 / 度")
ax.set_ylabel("速度 / (m/s)")
ax.legend(loc="upper right", fontsize=8)
fig.colorbar(cf, ax=ax, label="遮蔽时长 / s")
fig.savefig(os.path.join(FIG_DIR, "p2_航向速度等高线.png"), bbox_inches="tight"
, dpi=220)
plt.close(fig)

# ---- 图4: 最优遮蔽判据曲线 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(ts, maxdist, color=C_DATA, lw=1.8, label="云心到视线束最大距离")
ax.axhline(sc.CLOUD_R, color=C_THR, lw=1.6, ls="--", label="遮蔽阈值 10 m")
ax.fill_between(ts, 0, sc.CLOUD_R, where=mask, color=C_FILL, alpha=0.25, label=
"有效遮蔽区间")
ax.set_xlabel("时间 t / s")
ax.set_ylabel("最大距离 / m")
ax.set_ylim(0, max(30, maxdist.max() * 1.05))
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p2_最优遮蔽曲线.png"), bbox_inches="tight",
dpi=220)

```

```

plt.close(fig)

print("图片与结果已保存。")

if __name__ == "__main__":
    main()

```

Listing 3: 问题二：单弹投放优化求解代码

```

# -*- coding: utf-8 -*-
# 问题三：FY1投放3枚烟幕弹干扰M1，优化投放策略最大化遮蔽时间并集，导出result1.xlsx
import os
import sys
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize

warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = [
    "STHeiti", "SimHei", "Heiti TC", "Arial Unicode MS",
    "Hiragino Sans GB", "PingFang SC", "Microsoft YaHei",
    "Songti SC", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 220
plt.rcParams["savefig.dpi"] = 220

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FIG_DIR = os.path.join(BASE_DIR, "图片")
OUT_DIR = os.path.join(BASE_DIR, "结果")
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)
sys.path.insert(0, os.path.dirname(BASE_DIR))

import smoke_core as sc

UAV, MISSILE = "FY1", "M1"
COLORS = ["#1f77b4", "#ff7f0e", "#2ca02c"]
C_THR, C_UNION = "#d62728", "#9467bd"

```

```

def make_bombs(params):
    heading, speed, t1, g12, g23, d1, d2, d3 = params
    tds = [t1, t1 + g12, t1 + g12 + g23]
    delays = [d1, d2, d3]
    dets, tdet, drops = [], [], []
    for t_drop, delay in zip(tds, delays):
        drop, det, td = sc.bomb_points(UAV, heading, speed, t_drop, delay)
        drops.append(drop)
        dets.append(det)
        tdet.append(td)
    return drops, dets, tdet

def eval_all(params, dt=0.04):
    """一次网格遍历返回 (并集时长, [各弹单独时长])。"""
    _, dets, tds = make_bombs(params)
    if any(d[2] <= 0 for d in dets):
        return 0.0, [0.0, 0.0, 0.0]
    hit = sc.missile_hit_time(MISSILE)
    t_lo = min(tds)
    t_hi = min(max(td + sc.CLOUD_LIFE for td in tds), hit)
    if t_hi <= t_lo:
        return 0.0, [0.0, 0.0, 0.0]
    ts = np.arange(t_lo, t_hi + dt, dt)
    union = np.zeros(len(ts), dtype=bool)
    per = []
    for det, td in zip(dets, tds):
        m = sc.bomb_mask(MISSILE, det, td, ts)
        per.append(float(m.sum() * dt))
        union |= m
    return float(union.sum() * dt), per

def eval_union(params, dt=0.04):
    return eval_all(params, dt)[0]

_best = {"val": -1.0, "x": None, "hist": []}

def neg_obj(params):
    u, per = eval_all(params, dt=0.04)
    if u > _best["val"]:
        _best["val"] = u
        _best["x"] = np.array(params, float)
    if u > 0:
        # 鼓励三枚弹均有效贡献, 抑制"浪费弹"

```

```

        score = u + 0.3 * min(per)
    else:
        _, dets, tds = make_bombs(params)
        mind = 0.0
        for det, td in zip(dets, tds):
            hit = sc.missile_hit_time(MISSILE)
            t1 = min(td + sc.CLOUD_LIFE, hit)
            ts = np.arange(td, t1 + 0.1, 0.1)
            C = sc.cloud_center_array(det, td, ts)
            A = sc.missile_pos_array(MISSILE, ts)
            mind += sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1).min()
        score = -mind * 1e-3
    return -score

def main():
    bounds = [(2.4, 3.9), (70.0, 140.0), (0.0, 6.0),
              (1.0, 8.0), (1.0, 8.0), (1.0, 10.0), (1.0, 10.0), (1.0, 10.0)]
    rng = np.random.default_rng(11)
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    S = 120
    init = rng.uniform(lo, hi, size=(S, 8))
    init[0] = [np.pi, 96.8, 0.05, 1.0, 1.0, 2.9, 2.9, 2.9]
    init[1] = [np.pi, 120.0, 0.1, 2.5, 2.5, 3.6, 3.0, 2.5]
    init[2] = [np.pi, 140.0, 0.1, 3.0, 3.0, 5.0, 4.0, 3.0]
    init[3] = [np.pi, 140.0, 0.1, 4.0, 4.0, 3.5, 3.0, 2.5]

    def cb(xk, convergence):
        _best["hist"].append(_best["val"])

    result = differential_evolution(
        neg_obj, bounds, maxiter=150, tol=1e-7,
        mutation=(0.5, 1.0), recombination=0.7, seed=11,
        polish=False, callback=cb, workers=1, updating="deferred", init=init,
    )
    ref = minimize(lambda p: -eval_union(p, dt=0.01), _best["x"], method="Nelder-
Mead",
                  options={"xatol": 1e-3, "fatol": 1e-4, "maxiter": 800})
    cand = [result.x, _best["x"], ref.x]
    best = max(cand, key=lambda p: eval_union(p, dt=0.005))

    drops, dets, tds = make_bombs(best)
    heading, speed = best[0], best[1]
    total, ts, union = sc.union_time(MISSILE, dets, tds, dt=0.002)
    union_intervals = sc.interval_from_mask(ts, union)
    per_bomb = [sc.single_bomb_time(MISSILE, d, t, dt=0.002) for d, t in zip(dets,

```

```

tds)])

print("=" * 60)
print("问题三 求解（3弹并集优化）")
print(f"航向角={np.degrees(heading):.4f} 度，速度={speed:.4f} m/s")
for i in range(3):
    print(f"  弹{i+1}: 投放={np.round(drops[i],2)} 起爆={np.round(dets[i],2)} "
          f"t_d={tds[i]:.3f}s 单弹时长={per_bomb[i]:.3f}s")
print(f"并集有效遮蔽时长={total:.4f} s")
print(f"并集区间={[round(a,3),round(b,3)] for a,b in union_intervals}]")

# ---- result1.xlsx ----
rows = []
for i in range(3):
    rows.append({
        "无人机运动方向(度)": round(np.degrees(heading), 4),
        "无人机运动速度(m/s)": round(speed, 4),
        "烟幕干扰弹编号": i + 1,
        "投放点x(m)": round(drops[i][0], 3),
        "投放点y(m)": round(drops[i][1], 3),
        "投放点z(m)": round(drops[i][2], 3),
        "起爆点x(m)": round(dets[i][0], 3),
        "起爆点y(m)": round(dets[i][1], 3),
        "起爆点z(m)": round(dets[i][2], 3),
        "有效干扰时长(s)": round(per_bomb[i], 3),
    })
df = pd.DataFrame(rows)
df.to_excel(os.path.join(OUT_DIR, "result1.xlsx"), index=False)
df.to_csv(os.path.join(OUT_DIR, "问题3_投放策略.csv"), index=False, encoding="
utf-8-sig")
with open(os.path.join(OUT_DIR, "问题3_并集时长.txt"), "w", encoding="utf-8")
as f:
    f.write(f"并集有效遮蔽时长 = {total:.4f} s\n")
    f.write(f"并集区间 = {union_intervals}\n")

# ---- 图1: 遮蔽时间甘特图 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
for i, (d, t) in enumerate(zip(dets, tds)):
    hit = sc.missile_hit_time(MISSILE)
    tsi = np.arange(t, min(t + sc.CLOUD_LIFE, hit) + 0.002, 0.002)
    mi = sc.bomb_mask(MISSILE, d, t, tsi)
    for a, b in sc.interval_from_mask(tsi, mi):
        ax.barh(i + 1, b - a, left=a, height=0.5, color=COLORS[i], alpha=0.85)
for a, b in union_intervals:
    ax.barh(0, b - a, left=a, height=0.5, color=C_UNION, alpha=0.85)
ax.set_yticks([0, 1, 2, 3])
ax.set_yticklabels(["并集", "弹1", "弹2", "弹3"])

```

```

ax.set_xlabel("时间 t / s")
ax.grid(True, axis="x", color="#e6e6e6", lw=0.6)
fig.savefig(os.path.join(FIG_DIR, "p3_遮蔽甘特图.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图2: 三弹遮蔽判据曲线 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
for i, (d, t) in enumerate(zip(dets, tds)):
    hit = sc.missile_hit_time(MISSILE)
    tsi = np.arange(t, min(t + sc.CLOUD_LIFE, hit) + 0.01, 0.01)
    C = sc.cloud_center_array(d, t, tsi)
    A = sc.missile_pos_array(MISSILE, tsi)
    md = sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1)
    ax.plot(tsi, md, color=COLORS[i], lw=1.6, label=f"弹{i+1}")
ax.axhline(sc.CLOUD_R, color=C_THR, lw=1.6, ls="--", label="遮蔽阈值 10 m")
for a, b in union_intervals:
    ax.axvspan(a, b, color="#2ca02c", alpha=0.15)
ax.set_xlabel("时间 t / s")
ax.set_ylabel("云心到视线束最大距离 / m")
ax.set_ylim(0, 40)
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p3_三弹遮蔽曲线.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图3: 俯视几何 ----
tt = np.linspace(0, 40, 300)
Mtraj = sc.missile_pos_array(MISSILE, tt)
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.plot(Mtraj[:, 0], Mtraj[:, 1], color="#1f77b4", lw=2, label="导弹M1航迹")
fy = sc.UAV_INIT[UAV]
ax.plot([fy[0], drops[-1][0]], [fy[1], drops[-1][1]], color="#ff7f0e", lw=2,
label="FY1飞行段")
for i in range(3):
    ax.scatter([dets[i][0]], [dets[i][1]], color=COLORS[i], s=70, marker="*",
zorder=5, label=f"起爆点{i+1}")
    ax.add_patch(plt.Circle((dets[i][0], dets[i][1]), sc.CLOUD_R, color=COLORS[
i], alpha=0.25))
ax.scatter([sc.TARGET_BOTTOM[0]], [sc.TARGET_BOTTOM[1]], color="#8c564b", s=80,
marker="s", label="真目标")
ax.scatter([0], [0], color="black", s=45, marker="x", label="假目标")
ax.set_xlabel("x / m")
ax.set_ylabel("y / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=7)

```

```

fig.savefig(os.path.join(FIG_DIR, "p3_俯视几何.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

# ---- 图4: 单弹贡献与并集对比 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
labels = ["弹1", "弹2", "弹3", "并集"]
vals = per_bomb + [total]
bars = ax.bar(labels, vals, color=COLORS + [C_UNION], alpha=0.9)
ax.axhline(sum(per_bomb), color=C_THR, lw=1.4, ls="--", label=f"单弹时长之和={
sum(per_bomb):.2f}s")
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.1, f"{v:.2f}", ha="center",
    fontsize=9)
ax.set_ylabel("有效遮蔽时长 / s")
ax.grid(True, axis="y", color="#e6e6e6", lw=0.6)
ax.legend(loc="upper left", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p3_贡献对比.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

print("图片与结果已保存。")

if __name__ == "__main__":
    main()

```

Listing 4: 问题三：单机三弹接力优化求解代码

```

# -*- coding: utf-8 -*-
# 问题四：FY1、FY2、FY3各投1枚烟幕弹干扰M1，联合优化最大化遮蔽并集，导出result2.
xlsx
import os
import sys
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize

warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = [
    "STHeiti", "SimHei", "Heiti TC", "Arial Unicode MS",
    "Hiragino Sans GB", "PingFang SC", "Microsoft YaHei",

```

```

    "Songti SC", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 220
plt.rcParams["savefig.dpi"] = 220

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FIG_DIR = os.path.join(BASE_DIR, "图片")
OUT_DIR = os.path.join(BASE_DIR, "结果")
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)
sys.path.insert(0, os.path.dirname(BASE_DIR))

import smoke_core as sc

UAVS, MISSILE = ["FY1", "FY2", "FY3"], "M1"
COLORS = ["#1f77b4", "#ff7f0e", "#2ca02c"]
C_THR, C_UNION = "#d62728", "#9467bd"

def make_bombs(params):
    drops, dets, tds = [], [], []
    for k, u in enumerate(UAVS):
        heading, speed, t_drop, delay = params[4 * k:4 * k + 4]
        drop, det, td = sc.bomb_points(u, heading, speed, t_drop, delay)
        drops.append(drop)
        dets.append(det)
        tds.append(td)
    return drops, dets, tds

def eval_all(params, dt=0.04):
    _, dets, tds = make_bombs(params)
    if any(d[2] <= 0 for d in dets):
        return 0.0, [0.0, 0.0, 0.0]
    hit = sc.missile_hit_time(MISSILE)
    t_lo = min(tds)
    t_hi = min(max(td + sc.CLOUD_LIFE for td in tds), hit)
    if t_hi <= t_lo:
        return 0.0, [0.0, 0.0, 0.0]
    ts = np.arange(t_lo, t_hi + dt, dt)
    union = np.zeros(len(ts), dtype=bool)
    per = []
    for det, td in zip(dets, tds):
        m = sc.bomb_mask(MISSILE, det, td, ts)
        per.append(float(m.sum() * dt))
    union |= m

```

```

    return float(union.sum() * dt), per

def eval_union(params, dt=0.04):
    return eval_all(params, dt)[0]

_best = {"val": -1.0, "x": None, "hist": []}

def neg_obj(params):
    u, per = eval_all(params, dt=0.04)
    if u > _best["val"]:
        _best["val"] = u
        _best["x"] = np.array(params, float)
    if u > 0:
        score = u + 0.12 * sum(per)
    else:
        _, dets, tds = make_bombs(params)
        mind = 0.0
        for det, td in zip(dets, tds):
            hit = sc.missile_hit_time(MISSILE)
            t1 = min(td + sc.CLOUD_LIFE, hit)
            ts = np.arange(td, t1 + 0.1, 0.1)
            C = sc.cloud_center_array(det, td, ts)
            A = sc.missile_pos_array(MISSILE, ts)
            mind += sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1).min()
        score = -mind * 1e-4
    return -score

def uav_heading_to_origin(u):
    p = sc.UAV_INIT[u]
    return np.arctan2(-p[1], -p[0])

def main():
    bounds = []
    for u in UAVS:
        h0 = uav_heading_to_origin(u)
        bounds += [(h0 - 0.8, h0 + 0.8), (70.0, 140.0), (0.0, 25.0), (1.0, 14.0)]
    rng = np.random.default_rng(23)
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    S = 132
    init = rng.uniform(lo, hi, size=(S, 12))
    init[0] = [np.pi, 100.0, 0.05, 2.9, uav_heading_to_origin("FY2"), 140.0, 3.0,

```

```

5.0,
    uav_heading_to_origin("FY3"), 140.0, 5.0, 6.0]
init[1] = [np.pi, 140.0, 0.1, 3.6, uav_heading_to_origin("FY2"), 130.0, 6.0,
6.0,
    uav_heading_to_origin("FY3"), 140.0, 10.0, 6.0]

def cb(xk, convergence):
    _best["hist"].append(_best["val"])

result = differential_evolution(
    neg_obj, bounds, maxiter=130, tol=1e-7,
    mutation=(0.5, 1.0), recombination=0.7, seed=23,
    polish=False, callback=cb, workers=1, updating="deferred", init=init,
)
ref = minimize(lambda p: -eval_union(p, dt=0.01), _best["x"], method="Nelder-
Mead",
    options={"xatol": 1e-3, "fatol": 1e-4, "maxiter": 1500})
cand = [result.x, _best["x"], ref.x]
best = max(cand, key=lambda p: eval_union(p, dt=0.005))

drops, dets, tds = make_bombs(best)
total, ts, union = sc.union_time(MISSILE, dets, tds, dt=0.002)
union_intervals = sc.interval_from_mask(ts, union)
per_bomb = [sc.single_bomb_time(MISSILE, d, t, dt=0.002) for d, t in zip(dets,
tds)]

def nd(h):
    return float(np.degrees(h) % 360.0)

print("=" * 60)
print("问题四 求解（三机联合，各1弹）")
for k, u in enumerate(UAVS):
    heading, speed = best[4 * k], best[4 * k + 1]
    print(f" {u}: 航向={nd(heading):.3f}度 速度={speed:.3f} "
        f"投放={np.round(drops[k],2)} 起爆={np.round(dets[k],2)} "
        f"t_d={tds[k]:.3f}s 单弹={per_bomb[k]:.3f}s")
print(f"并集有效遮蔽时长={total:.4f} s")
print(f"并集区间={[round(a,3),round(b,3)] for a,b in union_intervals}")

rows = []
for k, u in enumerate(UAVS):
    heading, speed = best[4 * k], best[4 * k + 1]
    rows.append({
        "无人机编号": u,
        "无人机运动方向(度)": round(nd(heading), 4),
        "无人机运动速度(m/s)": round(speed, 4),
        "投放点x(m)": round(drops[k][0], 3),
    })

```

```

        "投放点y(m)": round(drops[k][1], 3),
        "投放点z(m)": round(drops[k][2], 3),
        "起爆点x(m)": round(dets[k][0], 3),
        "起爆点y(m)": round(dets[k][1], 3),
        "起爆点z(m)": round(dets[k][2], 3),
        "有效干扰时长(s)": round(per_bomb[k], 3),
    })
df = pd.DataFrame(rows)
df.to_excel(os.path.join(OUT_DIR, "result2.xlsx"), index=False)
df.to_csv(os.path.join(OUT_DIR, "问题4_投放策略.csv"), index=False, encoding="
utf-8-sig")
with open(os.path.join(OUT_DIR, "问题4_并集时长.txt"), "w", encoding="utf-8")
as f:
    f.write(f"并集有效遮蔽时长 = {total:.4f} s\n并集区间 = {union_intervals}\n"
)

# ---- 图1: 俯视几何 ----
tt = np.linspace(0, 50, 320)
Mtraj = sc.missile_pos_array(MISSILE, tt)
fig, ax = plt.subplots(figsize=(7.6, 5.4))
ax.plot(Mtraj[:, 0], Mtraj[:, 1], color="#d62728", lw=2, label="导弹M1航迹")
for k, u in enumerate(UAVS):
    fy = sc.UAV_INIT[u]
    ax.plot([fy[0], drops[k][0]], [fy[1], drops[k][1]], color=COLORS[k], lw=2)
    ax.scatter([fy[0]], [fy[1]], color=COLORS[k], s=45, marker="o")
    ax.scatter([dets[k][0]], [dets[k][1]], color=COLORS[k], s=75, marker="*",
               zorder=5, label=f"{u}起爆点")
    ax.add_patch(plt.Circle((dets[k][0], dets[k][1]), sc.CLOUD_R, color=COLORS[
k], alpha=0.3))
ax.scatter([sc.TARGET_BOTTOM[0]], [sc.TARGET_BOTTOM[1]], color="#8c564b", s=80,
           marker="s", label="真目标")
ax.scatter([0], [0], color="black", s=45, marker="x", label="假目标")
ax.set_xlabel("x / m")
ax.set_ylabel("y / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=7)
fig.savefig(os.path.join(FIG_DIR, "p4_俯视几何.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

# ---- 图2: 遮蔽甘特图 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
for k, (d, t) in enumerate(zip(dets, tds)):
    hit = sc.missile_hit_time(MISSILE)
    tsi = np.arange(t, min(t + sc.CLOUD_LIFE, hit) + 0.002, 0.002)
    mi = sc.bomb_mask(MISSILE, d, t, tsi)
    for a, b in sc.interval_from_mask(tsi, mi):

```

```

        ax.barh(k + 1, b - a, left=a, height=0.5, color=COLORS[k], alpha=0.85)
for a, b in union_intervals:
    ax.barh(0, b - a, left=a, height=0.5, color=C_UNION, alpha=0.85)
ax.set_yticks([0, 1, 2, 3])
ax.set_yticklabels(["并集"] + UAVS)
ax.set_xlabel("时间 t / s")
ax.grid(True, axis="x", color="#e6e6e6", lw=0.6)
fig.savefig(os.path.join(FIG_DIR, "p4_遮蔽甘特图.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图3: 遮蔽判据曲线 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
for k, (d, t) in enumerate(zip(dets, tds)):
    hit = sc.missile_hit_time(MISSILE)
    tsi = np.arange(t, min(t + sc.CLOUD_LIFE, hit) + 0.01, 0.01)
    C = sc.cloud_center_array(d, t, tsi)
    A = sc.missile_pos_array(MISSILE, tsi)
    md = sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1)
    ax.plot(tsi, md, color=COLORS[k], lw=1.6, label=UAVS[k])
ax.axhline(sc.CLOUD_R, color=C_THR, lw=1.6, ls="--", label="遮蔽阈值 10 m")
for a, b in union_intervals:
    ax.axvspan(a, b, color="#2ca02c", alpha=0.15)
ax.set_xlabel("时间 t / s")
ax.set_ylabel("云心到视线束最大距离 / m")
ax.set_ylim(0, 60)
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=8)
fig.savefig(os.path.join(FIG_DIR, "p4_遮蔽曲线.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

# ---- 图4: 贡献对比 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
labels = UAVS + ["并集"]
vals = per_bomb + [total]
bars = ax.bar(labels, vals, color=COLORS + [C_UNION], alpha=0.9)
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.05, f"{v:.2f}", ha="center",
fontsize=9)
ax.set_ylabel("有效遮蔽时长 / s")
ax.grid(True, axis="y", color="#e6e6e6", lw=0.6)
fig.savefig(os.path.join(FIG_DIR, "p4_贡献对比.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

print("图片与结果已保存。")

```

```
if __name__ == "__main__":
    main()
```

Listing 5: 问题四：三机协同优化求解代码

```
# -*- coding: utf-8 -*-
# 问题五：5架无人机各至多3弹，干扰M1/M2/M3。任务分配+分组贪心边际优化，导出result3.
    xlsx
import os
import sys
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution

warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = [
    "STHeiti", "SimHei", "Heiti TC", "Arial Unicode MS",
    "Hiragino Sans GB", "PingFang SC", "Microsoft YaHei",
    "Songti SC", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 220
plt.rcParams["savefig.dpi"] = 220

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FIG_DIR = os.path.join(BASE_DIR, "图片")
OUT_DIR = os.path.join(BASE_DIR, "结果")
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)
sys.path.insert(0, os.path.dirname(BASE_DIR))

import smoke_core as sc

UAVS = ["FY1", "FY2", "FY3", "FY4", "FY5"]
MISSILES = ["M1", "M2", "M3"]
MCOLOR = {"M1": "#1f77b4", "M2": "#ff7f0e", "M3": "#2ca02c"}
UCOLOR = {"FY1": "#1f77b4", "FY2": "#ff7f0e", "FY3": "#2ca02c",
          "FY4": "#d62728", "FY5": "#9467bd"}
```

```

def heading_to_origin(u):
    p = sc.UAV_INIT[u]
    return np.arctan2(-p[1], -p[0])

def single_best(uav, missile, seed=0, maxiter=35, S=40):
    """引导式差分进化估计单弹对某导弹的最优可达遮蔽时长，用于任务分配打分。"""
    h0 = heading_to_origin(uav)
    bounds = [(h0 - 1.0, h0 + 1.0), (70.0, 140.0), (0.0, 40.0), (1.0, 14.0)]
    hit = sc.missile_hit_time(missile)
    st = {"v": -1.0, "x": None}

    def evalf(p, dt=0.06):
        _, det, td = sc.bomb_points(uav, p[0], p[1], p[2], p[3])
        if det[2] <= 0:
            return 0.0, 1e6
        t1 = min(td + sc.CLOUD_LIFE, hit)
        if t1 <= td:
            return 0.0, 1e6
        ts = np.arange(td, t1 + dt, dt)
        C = sc.cloud_center_array(det, td, ts)
        A = sc.missile_pos_array(missile, ts)
        md = sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1)
        return float((md <= sc.CLOUD_R).sum() * dt), float(md.min())

    def neg(p):
        dur, mind = evalf(p)
        if dur > st["v"]:
            st["v"] = dur
            st["x"] = np.array(p, float)
        return -dur if dur > 0 else mind * 1e-3

    rng = np.random.default_rng(seed)
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    init = rng.uniform(lo, hi, size=(S, 4))
    init[0] = [h0, 130.0, 0.1, 4.0]
    init[1] = [h0, 100.0, 2.0, 5.0]
    differential_evolution(neg, bounds, maxiter=maxiter, tol=1e-6, seed=seed,
                           mutation=(0.5, 1.0), recombination=0.7,
                           polish=False, workers=1, updating="deferred", init=init)
    return max(st["v"], 0.0), st["x"]

def bombs_from(uav, missile, params, nb):
    """由参数生成 nb 枚弹。params: heading, speed, t1, [gaps...], [delays...]。"""
    heading, speed = params[0], params[1]

```

```

t1 = params[2]
gaps = params[3:3 + (nb - 1)]
delays = params[3 + (nb - 1):3 + (nb - 1) + nb]
tds_drop = [t1]
for g in gaps:
    tds_drop.append(tds_drop[-1] + g)
dets, tds, drops = [], [], []
for t_drop, delay in zip(tds_drop, delays):
    drop, det, td = sc.bomb_points(uav, heading, speed, t_drop, delay)
    drops.append(drop)
    dets.append(det)
    tds.append(td)
return drops, dets, tds

def coverage_mask(missile, dets, tds, ts):
    union = np.zeros(len(ts), dtype=bool)
    for det, td in zip(dets, tds):
        union |= sc.bomb_mask(missile, det, td, ts)
    return union

def optimize_uav(uav, missile, nb, fixed_dets, fixed_tds, seed, maxiter=60, S=48,
                 seed_params=None):
    """优化某UAV对某导弹投nb枚弹的边际遮蔽增量（相对已固定的其它云团）。"""
    h0 = heading_to_origin(uav)
    bounds = [(h0 - 0.9, h0 + 0.9), (70.0, 140.0), (0.0, 40.0)]
    bounds += [(1.0, 8.0)] * (nb - 1)
    bounds += [(1.0, 14.0)] * nb
    dt = 0.05
    hit = sc.missile_hit_time(missile)

    state = {"val": -1.0, "x": None}

    def grid_for(tds_all):
        t_lo = min(tds_all)
        t_hi = min(max(t + sc.CLOUD_LIFE for t in tds_all), hit)
        if t_hi <= t_lo:
            return None
        return np.arange(t_lo, t_hi + dt, dt)

    def marginal(params):
        _, dets, tds = bombs_from(uav, missile, params, nb)
        if any(d[2] <= 0 for d in dets):
            return -1e3
        all_tds = list(tds) + list(fixed_tds)
        ts = grid_for(all_tds)

```

```

    if ts is None:
        return -1e3
    base = coverage_mask(missile, fixed_dets, fixed_tds, ts)
    new = coverage_mask(missile, dets, tds, ts)
    gained = float((new & ~base).sum() * dt)
    return gained

def neg(params):
    g = marginal(params)
    if g > state["val"]:
        state["val"] = g
        state["x"] = np.array(params, float)
    # 平滑引导：无增益时用最小距离
    if g <= 0:
        _, dets, tds = bombs_from(uav, missile, params, nb)
        mind = 0.0
        for det, td in zip(dets, tds):
            t1 = min(td + sc.CLOUD_LIFE, hit)
            if t1 <= td:
                mind += 1e3
                continue
            tt = np.arange(td, t1 + 0.15, 0.15)
            C = sc.cloud_center_array(det, td, tt)
            A = sc.missile_pos_array(missile, tt)
            mind += sc._seg_point_dist(A, C, sc.TARGET_PTS).max(axis=1).min()
        return mind * 1e-4
    return -g

rng = np.random.default_rng(seed)
lo = np.array([b[0] for b in bounds])
hi = np.array([b[1] for b in bounds])
init = rng.uniform(lo, hi, size=(S, len(bounds)))
init[0, 0] = h0
init[0, 1] = 130.0
if seed_params is not None:
    h, sp, t_drop, delay = seed_params
    for r, gap in ((1, 1.5), (2, 2.5)):
        vec = [h, sp, t_drop] + [gap] * (nb - 1) + [delay] * nb
        init[r] = np.clip(np.array(vec, float), lo, hi)
differential_evolution(neg, bounds, maxiter=maxiter, tol=1e-7,
                       mutation=(0.5, 1.0), recombination=0.7, seed=seed,
                       polish=False, workers=1, updating="deferred", init=init)
return state["x"], state["val"]

def main():
    # ---- 1) 任务分配打分 ----

```

```

score = {}
score_params = {}
for i, u in enumerate(UAVS):
    for j, m in enumerate(MISSILES):
        v, p = single_best(u, m, seed=100 + 10 * i + j)
        score[(u, m)] = v
        score_params[(u, m)] = p
        print(f"单弹可达: {u}->{m} = {v:.3f} s")

# 均衡分配: 先保证每导弹分到其最强可达UAV, 再把剩余UAV按对各导弹的最高边际潜力
# 补充。
# 说明: M1威胁窗口长、可叠加多机而重叠饱和慢, 故剩余机按单弹可达打分择优归队。
assigned = {}
order = sorted(MISSILES, key=lambda m: max(score[(u, m)] for u in UAVS),
reverse=True)
for m in order:
    avail = [u for u in UAVS if u not in assigned]
    u = max(avail, key=lambda uu: score[(uu, m)])
    assigned[u] = m
for u in UAVS:
    if u not in assigned:
        assigned[u] = max(MISSILES, key=lambda m: score[(u, m)])
assign = {m: [] for m in MISSILES}
for u in UAVS:
    assign[assigned[u]].append(u)
print("任务分配:", assign)

# ---- 2) 分组贪心多弹优化 ----
plan = [] # 每弹: dict(uav, missile, drop, det, td, dur)
per_missile_dets = {m: [] for m in MISSILES}
per_missile_tds = {m: [] for m in MISSILES}
uav_params = {}
for m in MISSILES:
    # 组内UAV按单弹可达降序, 逐个做边际优化
    for order_i, u in enumerate(sorted(assign[m], key=lambda x: -score[(x, m)])):
        nb = 3 if score[(u, m)] > 1.0 else (2 if score[(u, m)] > 0.3 else 1)
        det_seed = 300 + 13 * UAVS.index(u) + 7 * MISSILES.index(m)
        x, gain = optimize_uav(u, m, nb, per_missile_dets[m], per_missile_tds[m],
                                seed=det_seed,
                                maxiter=70 if nb >= 2 else 45, S=52,
                                seed_params=score_params[(u, m)])
        if x is None or gain <= 0.05:
            print(f"{u}->{m}: 无有效增益(gain={0 if x is None else gain:.3f}),
不投弹")
            continue

```

```

_, dets, tds = bombs_from(u, m, x, nb)
uav_params[u] = (x, nb, m)
for k in range(nb):
    per_missile_dets[m].append(dets[k])
    per_missile_tds[m].append(tds[k])
print(f"{u}->{m}: nb={nb}, 边际增益={gain:.3f}s")

# ---- 3) 汇总每导弹并集与每弹单独时长 ----
total = 0.0
missile_union = {}
for m in MISSILES:
    if per_missile_dets[m]:
        u_time, ts, union = sc.union_time(m, per_missile_dets[m],
per_missile_tds[m], dt=0.002)
    else:
        u_time, ts, union = 0.0, None, None
    missile_union[m] = (u_time, ts, union)
    total += u_time
    print(f"导弹{m} 遮蔽并集时长 = {u_time:.4f} s")
print(f"三导弹遮蔽时长总和 = {total:.4f} s")

# 生成每弹记录（仅保留有效贡献弹，按无人机重新编号）
rows = []
for u in UAVS:
    if u not in uav_params:
        continue
    x, nb, m = uav_params[u]
    drops, dets, tds = bombs_from(u, m, x, nb)
    heading, speed = x[0], x[1]
    eff = []
    for k in range(nb):
        dur = sc.single_bomb_time(m, dets[k], tds[k], dt=0.002)
        if dur >= 0.05:
            eff.append((drops[k], dets[k], tds[k], dur))
    for idx, (drop, det, td, dur) in enumerate(eff):
        plan.append(dict(uav=u, missile=m, drop=drop, det=det, td=td, dur=dur))
    rows.append({
        "无人机编号": u,
        "无人机运动方向(度)": round(float(np.degrees(heading) % 360.0), 4),
        "无人机运动速度(m/s)": round(float(speed), 4),
        "烟幕干扰弹编号": idx + 1,
        "投放点x(m)": round(float(drop[0]), 3),
        "投放点y(m)": round(float(drop[1]), 3),
        "投放点z(m)": round(float(drop[2]), 3),
        "起爆点x(m)": round(float(det[0]), 3),
        "起爆点y(m)": round(float(det[1]), 3),
        "起爆点z(m)": round(float(det[2]), 3),
    })

```

```

        "干扰的导弹编号": m,
        "有效干扰时长(s)": round(float(dur), 3),
    })
df = pd.DataFrame(rows)
df.to_excel(os.path.join(OUT_DIR, "result3.xlsx"), index=False)
df.to_csv(os.path.join(OUT_DIR, "问题5_投放策略.csv"), index=False, encoding="
utf-8-sig")
pd.DataFrame([{"导弹": m, "遮蔽并集时长(s)": round(missile_union[m][0], 4)} for
m in MISSILES]
            + [{"导弹": "总和", "遮蔽并集时长(s)": round(total, 4)}]
            ).to_csv(os.path.join(OUT_DIR, "问题5_导弹遮蔽汇总.csv"), index=
False, encoding="utf-8-sig")

# 分配打分矩阵存档（供论文热力图）
smat = np.array([[score[(u, m)] for m in MISSILES] for u in UAVS])
pd.DataFrame(smat, index=UAVS, columns=MISSILES).to_csv(
    os.path.join(OUT_DIR, "问题5_单弹可达矩阵.csv"), encoding="utf-8-sig")

# ---- 图1：任务分配打分热力图 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
im = ax.imshow(smat, cmap="viridis", aspect="auto")
ax.set_xticks(range(len(MISSILES)))
ax.set_xticklabels(MISSILES)
ax.set_yticks(range(len(UAVS)))
ax.set_yticklabels(UAVS)
for i in range(len(UAVS)):
    for j in range(len(MISSILES)):
        ax.text(j, i, f"{smat[i,j]:.2f}", ha="center", va="center",
                color="white" if smat[i, j] < smat.max() * 0.6 else "black",
                fontsize=9)
ax.set_xlabel("来袭导弹")
ax.set_ylabel("无人机")
fig.colorbar(im, ax=ax, label="单弹可达遮蔽时长 / s")
fig.savefig(os.path.join(FIG_DIR, "p5_分配热力图.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图2：各导弹遮蔽时间轴 ----
fig, ax = plt.subplots(figsize=(7.6, 5.2))
row = 0
yticks, ylabels = [], []
for m in MISSILES:
    for p in [q for q in plan if q["missile"] == m]:
        hit = sc.missile_hit_time(m)
        tsi = np.arange(p["td"], min(p["td"] + sc.CLOUD_LIFE, hit) + 0.002,
0.002)
        mi = sc.bomb_mask(m, p["det"], p["td"], tsi)

```

```

        for a, b in sc.interval_from_mask(tsi, mi):
            ax.barh(row, b - a, left=a, height=0.55, color=UCOLOR[p["uav"]],
alpha=0.85)
            yticks.append(row)
            ylabels.append(f"{m}:{p['uav']}")
            row += 1
ax.set_yticks(yticks)
ax.set_yticklabels(ylabels, fontsize=8)
ax.set_xlabel("时间 t / s")
ax.grid(True, axis="x", color="#e6e6e6", lw=0.6)
fig.savefig(os.path.join(FIG_DIR, "p5_遮蔽时间轴.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图3: 各导弹并集时长柱状 ----
fig, ax = plt.subplots(figsize=(7.2, 5.2))
vals = [missile_union[m][0] for m in MISSILES]
bars = ax.bar(MISSILES, vals, color=[MCOLOR[m] for m in MISSILES], alpha=0.9)
ax.axhline(total, color="#9467bd", lw=1.4, ls="--", label=f"总和={total:.2f}s")
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.05, f"{v:.2f}", ha="center",
fontsize=10)
ax.set_ylabel("遮蔽并集时长 / s")
ax.legend(loc="upper right", fontsize=8)
ax.grid(True, axis="y", color="#e6e6e6", lw=0.6)
fig.savefig(os.path.join(FIG_DIR, "p5_导弹遮蔽柱状.png"), bbox_inches="tight",
dpi=220)
plt.close(fig)

# ---- 图4: 整体俯视态势 ----
fig, ax = plt.subplots(figsize=(7.8, 5.6))
tt = np.linspace(0, 60, 320)
for m in MISSILES:
    Mt = sc.missile_pos_array(m, tt)
    ax.plot(Mt[:, 0], Mt[:, 1], color=MCOLOR[m], lw=1.8, ls="-", label=f"导弹{m}
}")
    ax.scatter([sc.MISSILE_INIT[m][0]], [sc.MISSILE_INIT[m][1]], color=MCOLOR[m]
], s=40, marker="v")
for u in UAVS:
    p = sc.UAV_INIT[u]
    ax.scatter([p[0]], [p[1]], color=UCOLOR[u], s=45, marker="o")
    ax.annotate(u, (p[0], p[1]), fontsize=8, xytext=(4, 4), textcoords="offset
points")
for q in plan:
    ax.scatter([q["det"][0]], [q["det"][1]], color=UCOLOR[q["uav"]], s=55,
marker="*", zorder=5)
    ax.add_patch(plt.Circle((q["det"][0], q["det"][1]), sc.CLOUD_R, color=

```

```

UCOLOR[q["uav"]], alpha=0.25))
ax.scatter([sc.TARGET_BOTTOM[0]], [sc.TARGET_BOTTOM[1]], color="#8c564b", s=90,
marker="s", label="真目标")
ax.scatter([0], [0], color="black", s=45, marker="x", label="假目标")
ax.set_xlabel("x / m")
ax.set_ylabel("y / m")
ax.grid(True, color="#e6e6e6", lw=0.6)
ax.legend(loc="upper right", fontsize=7)
fig.savefig(os.path.join(FIG_DIR, "p5_整体态势.png"), bbox_inches="tight", dpi
=220)
plt.close(fig)

print("图片与结果已保存。")

if __name__ == "__main__":
    main()

```

Listing 6: 问题五：多机多弹多目标分配优化求解代码